

LLMs can construct powerful representations and streamline sample-efficient supervised learning

Ilker Demirel
MIT

Lawrence Shi
MIT

Zeshan Hussain
MIT, Harvard Medical School

David Sontag
MIT

 Code  Website

Abstract

As real-world datasets become more complex and heterogeneous, supervised learning is often bottlenecked by input representation design. Modeling multimodal data, such as time-series, free text, and structured records, often requires non-trivial domain expertise. We propose an agentic pipeline to streamline this process. First, an LLM analyzes a small but diverse subset of *text-serialized* input examples *in-context* to synthesize a *global rubric*, which acts as a programmatic specification for extracting and organizing evidence. This rubric is then used to transform *naive* text-serializations of inputs into a more standardized format for downstream models. We also describe *local rubrics*, which are task-conditioned interpretive summaries generated by an LLM. Across 15 clinical tasks from the EHRSHOT benchmark, our rubric approaches significantly outperform count-feature models, naive LLM baselines, and a clinical foundation model pretrained on orders of magnitude more data. Beyond performance, rubrics offer operational advantages such as being easy to audit, cost-effectiveness at scale, and facilitating tabular representations.

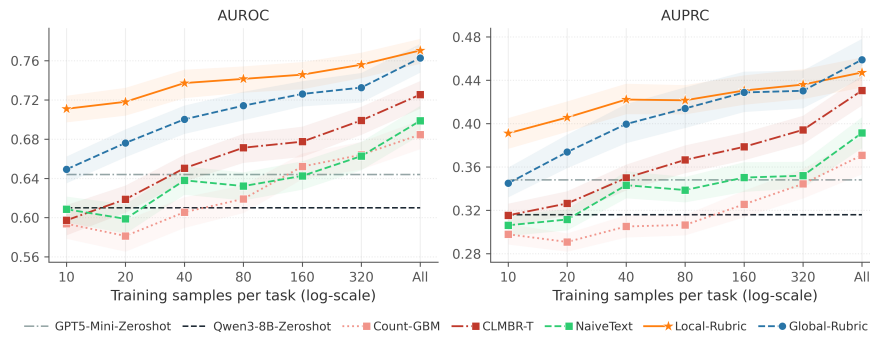


Figure 1: Performance (macro-averaged) over 15 clinical prediction tasks in the EHRSHOT benchmark [61], swept over training-set size n (per task). Our rubric-style representations constructed by LLMs outperform the *naive* text-serialization baseline in Hegselmann et al. [15], as well as a clinical foundation model pretrained on 2.57M patient timelines (CLMBR-T, [61]), and a count feature-based gradient boosting machine (Count-GBM, [21, 61]).

1 Introduction

Supervised learning underpins a wide range of applications across domains. In medicine, deep neural networks achieve specialist-level performance in pneumonia detection and diabetic retinopathy screening [12, 41]. In finance, credit risk assessment models outperform legacy scorecards [26]. In environmental science, supervised learning enables weather forecasting from radar observations [42].

Naive Text Representation	Local Rubric Representation	Global Rubric Representation
# Patient Demographics - Patient age: 78, Female [...] # Detailed Past Medical Visits ## Inpatient Visit (14 days to pred. time, current visit) ### Conditions - Acute posthemorrhagic anemia - pH meas., venous: 7.25, 7.31 [...] ### Medications - furosemide 20 MG Oral Tablet [...] ### Procedures - Chest x-ray - Electrocardiogram report [...] ## Emergency Room Visit (87 days before prediction time) ### Conditions - Benign essential hypertension - Chest pain [...]	1. Patient Snapshot - 27 yo hispanic male. - Recurrent cardiology visits for congenital anomaly of coronary artery [...] 2. Main Risk Factors - Congenital coronary artery anomaly (established structural predisposition to myocardial ischemia/infarction). - Tobacco exposure (smokeless tobacco reported) [...] 3. Protective Factors - Young age (27) — lower baseline atherosclerotic burden relative to older adults. - Normal BMI (21-22). - No documented diabetes (glucose in normal range) or chronic renal impairment [...]	3. Demographics - 55 Female [...] 6. Recent Cardiac Symptoms (last 365 days) - Chest pain/angina: No - Dyspnea/shortness of breath: Yes (date unknown) [...] 12. Other Relevant Labs - Creatinine: 1.12 (2023-12-02) - eGFR: No data [...] 17. Known Risk Factors - Diabetes: No (A1c date unknown) - Family history of premature CAD: Unknown [...] 20. Non-cardiac Serious Illness That May Mimic or Alter MI Risk - Active malignancy: No [...]

Figure 2: Synthetic electronic health record representation examples. *Left*. Naive text-serialization adopted from Hegselmann et al. [15]. *Middle*. Local rubric representation (task-conditioned summary of the naive text-serialization). *Right*. Global rubric transformed version of the naive-text serialization.

The success of supervised learning depends on the availability of input representations that can be easily processed by off-the-shelf models. Real-world datasets, however, are increasingly more complex and heterogeneous. They combine structured fields with unstructured text, time-series, and images. In healthcare, clinical prediction may benefit from longitudinal labs, coded events (*e.g.*, diagnoses), free-text notes, and medical images. In finance, stock-price forecasting and risk modeling may involve price and volume time-series, news reports, and structured events such as analyst ratings.

In domains with complex data, representation design requires bespoke engineering and domain expertise. Even then, resulting representations are not necessarily optimal: they may discard critical signal or bury it in noise. We show how large language models (LLMs) can build *agentic* pipelines that automate designing powerful input representations and enable sample-efficient supervised learning.

LLMs offer a practical interface to heterogeneous data through text-serializations. Song et al. [50] and Akhauri et al. [3] serialize diverse configurations and logs into text sequences to predict system performance metrics. Hegselmann et al. [15] serialize longitudinal electronic health records (EHR) into Markdown and train linear heads over text-embeddings for clinical prediction tasks (see Figure 2, *left*). Demirel et al. [8] use LLMs to predict daily user activities by combining multimodal time-series data from wearables after transforming them into textual descriptions.

These works show LLMs’ potential to streamline supervised learning with complex datasets, but they treat text-serialization of the input as fixed and leave the bulk of *calibration* to data for the downstream model. In contrast, we take the text-serialized input as a starting point and show how LLMs can automate constructing better representations to directly improve downstream performance.

Beyond modeling convenience, LLMs’ pretraining knowledge can enable effective regularization, which is key to sample-efficiency. Our work aligns with literature on injecting knowledge into statistical models: LMPriors uses language descriptions as task-specific priors [7], and TabLLM shows effective few-shot tabular learning [14]. LLM-Select and LLM-Lasso guide feature selection and regularization [17, 66], and Kim et al. [23] use task metadata to construct inductive biases.

These methods mostly use LLMs to augment traditional models working on clean datasets. In contrast, we focus on representation design: how complex inputs should be organized *prior* to downstream learning. In that sense, our work also aligns with recent literature on learning in the language space, such as GEPA by Agrawal et al. [1]. An extended related work section is included in Appendix A.

Our contribution. We propose *rubrics*, which are used to process complex text inputs into a standardized and information-rich format that can be easily and efficiently digested by downstream learners. We assume *naive* text-serializations of inputs are available, or they can be constructed straightforwardly (see Figure 2, left for an example). We develop two types of rubrics, which are given below and detailed in Section 2.

Global rubrics. A global rubric is a task-level specification that defines what information should be extracted from the input and how. It is *generated* by prompting an LLM with a diverse set of examples (see Figure 3, Panels A–C).

Local rubrics. We ask an LLM to produce a *task-conditioned* interpretive summary with structured sections (see Figure 4, left), similar to recent work on explainable clinical prediction models [39].

Advantages of global rubrics. Both rubrics achieve similar downstream performance and outperform the baselines. However, local rubrics do not have the same level of standardization as global rubrics, which endows the latter with several practical desiderata lacking in the former.

- *Auditable and improvable:* Global rubrics are more amenable to inspections by domain experts, such as for analyzing subgroup bias risk and iterative refinement.
- *More operationally useful:* Global rubric representations can be transformed into tabular features (Figure 3, Panel F), immediately unlocking a suite of machine learning techniques.
- *Cheaper to deploy:* Global rubric transformation at inference time can be automated (see Figure 3, Panels E and F), whereas summarization requires an LLM forward pass per example. This makes global rubrics “free” compared to local rubrics, which incur $\mathcal{O}(N)$ time and compute cost.

We evaluate on 15 binary clinical prediction tasks from the EHRSHOT benchmark [61], spanning operational outcomes, new diagnoses, lab results, and chest X-ray findings. We compare against a gradient boosting machine with count-based features (Count-GBM, [21]), a clinical foundation model pretrained on 2.57M patients (CLMBR-T, [61]), zero-shot chain-of-thought prompting (CoT) with Qwen3-8B and GPT5-Mini¹ [36, 40, 59], and the LLM baseline of Hegselmann et al. [15], which uses the naive EHR text-serializations we build on top of.

We preview the main findings here and provide detailed discussion in Section 5. Rubrics substantially outperform baselines on average across sample sizes n , with gaps largest for small n . First, the LLM’s *interpretation* of evidence acts as a sample-efficiency lever. Stripping it from local rubrics costs noticeable performance at small n and almost none at $n=\text{All}$, indicating that pretrained world knowledge supplies a prior the downstream classifier increasingly does without as data accumulate. Second, *standardized* global rubric templates are themselves strong representations even without an interpretive layer, beating all baselines across all sample sizes. Global rubrics trail local rubrics at small n , where the LLM-injected statistical prior in the language space matters most, but the gap closes by $n=\text{All}$ as labels accumulate.

2 Rubric representation learning with LLMs

2.1 Global rubrics

We introduce global rubrics for converting heterogeneous, weakly structured inputs into standardized, task-aligned representations. While we focus on electronic health records (EHR), the procedure applies wherever inputs can be text-serialized. Throughout, we use GPT5-Mini for natural-language steps (rubric synthesis and application, local-rubric generation) and GPT-5.2 for code-generation, as it produced more reliable scripts in our pilots (Figure 3, Panels E, F).

Setup and notation. We describe the global rubric learning procedure for a single prediction task. Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ denote labeled training data, where x is a *raw input* and $y \in \{0, 1\}$ is the task label. Let $s(\cdot)$ be some serialization procedure that maps an input x into its textual representation, and define $x^{\text{text}} = s(x)$. A rubric specifies a task-specific transformation

$$\mathcal{R} : x^{\text{text}} \mapsto x^{\text{rubric}},$$

where x^{rubric} is a more structured representation of the same underlying input x , and it can be used with downstream predictors instead of x^{text} . We describe downstream training in Section 4.

Global rubric synthesis. Global rubric learning has two stages, shown in Panels A and B of Figure 3. First, we select a small, label-balanced and *diverse* cohort from the training split. Second, an LLM inspects this cohort *in-context* and synthesizes a task-specific rubric by defining predictive features and describing how to extract them.

¹We used GPT5-Mini and GPT-5.2 via the HIPAA-compliant Microsoft Azure OpenAI Service.

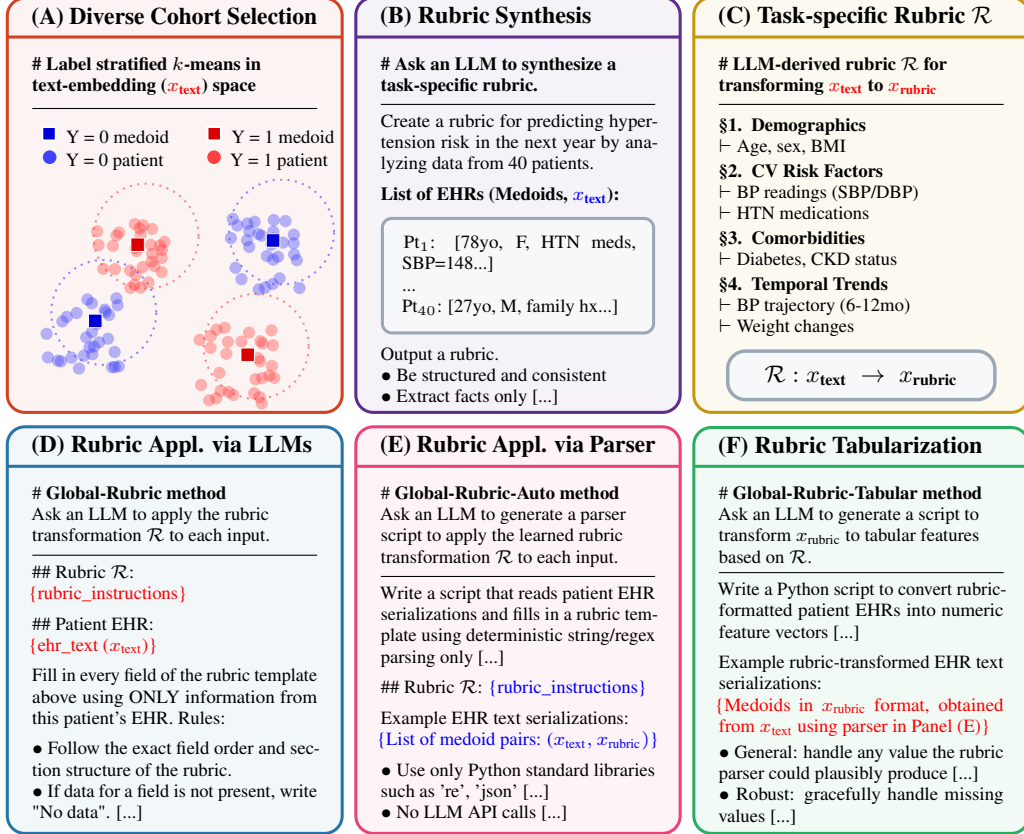


Figure 3: Agentic global-rubric pipeline for EHRSHOT tasks. Full prompts are in Appendix F.

- *Step 1) Diverse cohort selection:* Rubric synthesis is done through a single prompt to an LLM (GPT5-Mini). We first embed each text-serialized input x_i^{text} into a vector space using a pretrained text-embedding model [69], and stratify by label,

$$\mathcal{D}^+ = \{x_i^{\text{text}} : y_i = 1\}, \quad \mathcal{D}^- = \{x_i^{\text{text}} : y_i = 0\}.$$

We perform k -means clustering within each stratum, where k is the number of clusters per label-stratum, so the final cohort contains $2k$ examples. Due to context size limitations, we use $k = 20$. Finally, we take the cluster medoids to obtain a diverse cohort (see Figure 3, Panel A).

- *Step 2) Rubric synthesis:* Given the selected cohort, we ask an LLM (GPT5-Mini) to produce a task-specific rubric that (i) defines discriminative, task-relevant signals, and (ii) specifies how each signal should be extracted from a given input, x^{text} (see Figure 3, Panel B). The full prompt is provided in Appendix F.2, and two full-rubric examples can be found in Appendix G.

Global rubric application. A global rubric $\mathcal{R}$ is applied to naive text-serializations, x^{text} , to produce x^{rubric} . We propose four different methods at this stage.

- **Global-Rubric** (LLM application; Figure 3, Panel D): We prompt an LLM (GPT5-Mini) with the global rubric $\mathcal{R}$, and the naive text-serialization of the input x^{text} , asking it to return x^{rubric} .
- **Global-Rubric-Auto** (parser-script application; Figure 3, Panel E): We prompt an LLM (GPT-5.2) with $\mathcal{R}$ and 40 paired $(x^{\text{text}}, x^{\text{rubric}})$ examples, asking it to write a *deterministic* parser script that converts x^{text} to x^{rubric} . The script is then used at deployment time without LLM calls.
- **Global-Rubric-Tabular** (tabularization-script; Figure 3, Panel F): We prompt an LLM (GPT-5.2) with the global rubric $\mathcal{R}$, parser script for the rubric transformation (see item above), and some examples (40) of *parser-generated* rubric-transformations, x^{rubric} . We ask the LLM to write a script to convert x^{rubric} into a set of tabular features.
- **Global-Rubric-Blind:** $\mathcal{R}$ is generated from the task description and the LLM’s world knowledge alone (skipping Step 1 in rubric synthesis above), then applied like **Global-Rubric**.

# Local-Rubric prompt Read the EHR and write a reasoning trace that characterizes the patient’s risk profile for the task: {task_query} — START OF EHR DATA — {NaiveText Serialization (x^{text})} — END OF EHR DATA — Your output must follow the exact section structure given below: 1. Patient snapshot 2. Main risk factors 3. Protective factors 4. What’s unknown & can swing the risk 5. Weighing & aggregating the evidence	# Local-Rubric-NoInterp prompt Below is a clinical summary of an EHR. Produce a minimally edited version that removes interpretive language. — START OF EHR SUMMARY — {Local-Rubric Representation (x^{rubric})} — END OF EHR SUMMARY — Remove: - Interpretive words (e.g., ‘suggests’) - Predictions, risk assessments [...] Keep unchanged: - Facts: demographics, diagnoses, lab values with numbers, vitals, [...] - Exact wording of factual statements [...]	# Local-Rubric-Basic Prompt Read the EHR below. Extract and summarize the evidence that is relevant to the task: {task_query} — START OF EHR DATA — {NaiveText Serialization (x^{text})} — END OF EHR DATA — - ONLY list factual evidence in the EHR. - Do NOT interpret, weigh, or reason about the evidence. - Do NOT assess risk, draw conclusions, or make predictions. - Do NOT use language like “suggests”, “indicates”, “consistent with”, “increases risk”, or “protective”.
---	--	--

Figure 4: Prompts for generating local rubric representations. *Left.* Task-conditioned interpretive summaries (Local-Rubric). *Middle.* Edited summaries with interpretations stripped away (Local-Rubric-NoInterp, full prompt in Appendix F.6). *Right.* Fact-listing summaries (Local-Rubric-Basic).

2.2 Local rubrics

Global rubrics define a structure shared across *all* inputs, once applied. Beyond performance gains, this unlocks several practical advantages, as summarized in Section 1. However, it is important to characterize the effect of standardizing the input on statistical performance.

To study this, we introduce *local rubrics*: task-conditioned summaries of x^{text} , generated independently for each example by an LLM (GPT5-Mini). Unlike global rubrics, they impose only a general section-level structure, giving the LLM flexibility to extract and interpret the most relevant evidence per case. We define three variants that progressively ablate the LLM’s interpretation of the evidence:

- **Local-Rubric**: a task-conditioned summary that includes both factual evidence and the LLM’s reasoning over it (Figure 4, left).
- **Local-Rubric-NoInterp**: starts from **Local-Rubric** and strips out explicit interpretive or predictive language while preserving factual evidence and pointers to missing or otherwise notable unobserved information (Figure 4, middle).
- **Local-Rubric-Basic**: asks the LLM to extract only task-relevant facts, without weighing evidence, reasoning, or assessing risk (Figure 4, right). It is the simplest of the three and may omit useful cues retained by **Local-Rubric-NoInterp**, such as potential missingness flags.

3 EHRSHOT benchmark

We evaluate on EHRSHOT [61], a longitudinal EHR benchmark which contains deidentified data from 6,739 patients at Stanford Medicine, including demographics, diagnoses, procedures, medications, and labs across full patient timelines with millions of coded events.

Clinical prediction tasks. EHRSHOT comprises 15 binary classification tasks across 4 categories:

- *Operational outcomes*: ICU transfer, Long length of stay (> 7 days), 30-day readmission
- *Assignment of new diagnoses*: Acute myocardial infarction (MI), Celiac, Hyperlipidemia, Hypertension, Lupus, Pancreatic cancer
- *Anticipating labs*: Anemia, Hyperkalemia, Hypoglycemia, Hyponatremia, Thrombocytopenia
- *Chest X-ray*: Abnormal chest x-ray findings

Operational tasks predict near-term events in the context of the current visit. Diagnosis tasks predict new diagnoses within one year. Lab tasks predict abnormal upcoming results for the most recent lab order. The chest X-ray task predicts abnormal radiology findings.

Subsampling. Some methods use an LLM call per example, so we subsample some EHRSHOT tasks to keep training and evaluation budget within a reasonable boundary (full details provided in Section B). All methods, including the baselines, are trained and evaluated on the same subsampled splits. Section E.4 reports **Global-Rubric-Tabular** performance on the full EHRSHOT dataset, since it does not use LLM calls at deployment.

4 Methods and downstream training

4.1 Our rubric representation-based methods

In Section 2, we introduced six textual rubric variants—[Global-Rubric](#), [Global-Rubric-Blind](#), [Global-Rubric-Auto](#), [Local-Rubric](#), [Local-Rubric-NoInterp](#), and [Local-Rubric-Basic](#)—and one tabular variant, [Global-Rubric-Tabular](#). For all textual variants, the rubric output x^{rubric} (or x^{text} for [NaiveText](#)) is wrapped in a unified task-conditioned prompt template (Figure 15 in Appendix F.1) before being encoded into a numerical vector by a frozen pretrained text-embedding model, and an L2-regularized logistic regression classifier is fit on top of those embeddings. We report results with Qwen3-Embedding-8B [69] as the default backbone in the main paper; LLaMA-3-8B [5, 33], Mistral-7B [5, 18], and OpenAI’s text-embedding-3-large [35] are evaluated as additional backbones in Section E.2. For the tabular variant, the LLM-generated parser/tabularization scripts produce a feature vector that is fed into an XGBoost classifier [6]. Hyperparameter tuning details are provided in Section E.5.

4.2 EHRSHOT baselines

We include [Count-GBM](#) following EHRSHOT [61], where each EHR is converted into a vector of code counts observed prior to the prediction time. Different time-windows are used, such as last 90 days and 90-180 days before. A LightGBM classifier is then trained [21].

We also evaluate [CLMBR-T](#), a transformer-based autoregressive medical foundation model. It is pretrained with a next-code prediction objective, using longitudinal data from 2.57M patients drawn from the same distribution as the EHRSHOT dataset [61]. For downstream tasks, a logistic regression classifier is fit on top of the vector embeddings extracted from CLMBR-T.

4.3 LLM-based baselines

Our rubric representations, x^{rubric} , are derived from *naive* text-serializations of the input, x^{text} . We adopt the serialization introduced by Heggelmann et al. [15] (see Figure 2, left). Each record includes patient demographics, a “General Medical Events” section for codes that are not tied to any visit, and a “Detailed Past Medical Visits” section listing visits in reverse chronological order. We refer to the baseline that uses x^{text} directly as [NaiveText](#). As with the textual rubrics, x^{text} is embedded using a pretrained embedding model, and a logistic regression classifier is trained on top.

We also evaluate zero-shot chain-of-thought (CoT) prompting [59]. For each example, [Qwen3-8B](#) and [GPT5-Mini](#) are prompted to reason over the [NaiveText](#) EHR serialization and give a final Yes/No answer. We sample 10 responses and estimate the probability as the fraction of Yes answers.

5 Quantitative results

Rubrics outperform the baselines on average. We evaluate the 15 EHRSHOT tasks across training-set sizes from $n=10$ per class to $n=\text{All}$ (Figures 5 and 7, with numerical results in Table 2 in Appendix E.1, and per-task tables in Appendix E.7). We use 1:1 label-balanced training sets except for $n=\text{All}$. [Global-Rubric](#) and [Local-Rubric](#) outperform every baseline for all values of n , with the largest gap at small n . At $n=10$, [Global-Rubric](#) reaches 0.649 AUROC (0.345 AUPRC) and [Local-Rubric](#) 0.701 (0.382), while [NaiveText](#), [CLMBR-T](#), and [Count-GBM](#) all remain near 0.60 (0.30). At $n=\text{All}$, [Global-Rubric](#) obtains 0.763 (0.459) and [Local-Rubric](#) 0.772 (0.452), with the strongest baseline, [CLMBR-T](#), trailing at 0.725 (0.430). On rare-event tasks, rubric methods roughly double the baselines’ AUPRC (e.g., new diagnoses at $n=10$).

Sample-efficiency: gains are largest at small n . Rubric methods outpace traditional baselines most strongly when labels are scarce, and the gap narrows but does not close at $n=\text{All}$. [CLMBR-T](#), in particular, trails our rubric methods by more than 0.10 AUROC and ~ 0.07 AUPRC at $n=10$. It is pretrained on a domain-specific corpus of 2.57M patient records drawn from the same distribution as EHRSHOT, a form of prior knowledge no rubric method has access to. Rubrics nonetheless surpass it, drawing on a complementary source of prior knowledge from the general-purpose, web-scale pretraining of the LLMs used during rubric synthesis and application.

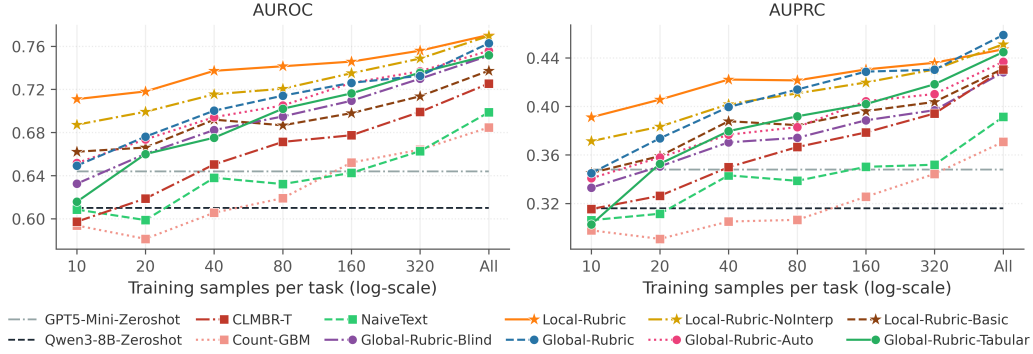


Figure 5: Average performance over all tasks. For confidence bands, see Appendix E.3.

Local-Rubric representation 72-year-old female. Serum sodium 138 → 134 → 131 mmol/L over 24 days, last value 5 days before prediction. Hydrochlorothiazide 25 mg daily started 12 days before prediction. Thiazide-induced hyponatremia is most pronounced in the first 1–2 weeks of initiation, raising the prior probability of an abnormal sodium. Mild congestive heart failure adds a chronic dilutional pathway. Renal function preserved (Cr 0.9 mg/dL), making a renal-failure mechanism unlikely. Documented fatigue and mild nausea at the most recent visit are consistent with early symptomatic hyponatremia. Overall: elevated prior risk.	Local-Rubric-NoInterp representation 72-year-old female. Serum sodium 138 → 134 → 131 mmol/L over 24 days, last value 5 days before prediction. Hydrochlorothiazide 25 mg daily started 12 days before prediction. Documented mild congestive heart failure. Renal function preserved (Cr 0.9 mg/dL). Documented fatigue and mild nausea at the most recent visit.	Global-Rubric representation Patient: Age 72, Sex FEMALE. ProblemListFlags: CKD/ESRD No, Dialysis No, Prior hyponatremia No, Active malignancy No. SerumSodium_Last3: 2024-08-10 (5d) 131; 2024-08-01 (14d) 134; 2024-07-22 (24d) 138 (mmol/L). SerumSodium_Min90: 131 mmol/L (2024-08-10). Meds affecting sodium: 2024-08-03 hydrochlorothiazide, thiazide diuretic, oral, 25 mg daily. RenalFunction: Cr 0.9 mg/dL, BUN 18 mg/dL (2024-08-10). Other fields: NA.
---	---	--

Figure 6: Three rubric outputs for the same synthetic hyponatremia case. **Local-Rubric** renders the evidence as a task-aligned risk assessment, **Local-Rubric-NoInterp** keeps the same evidence with interpretive language stripped, and **Global-Rubric** extracts it into standardized fields.

Input representation design is crucial to performance gains. **NaiveText** and our rubric methods share the same downstream stack (a pretrained text-embedding model with a logistic-regression head), so the gap between them isolates the value of LLM-driven representation design in the language space. At $n=10$ /All, **Local-Rubric** improves on **NaiveText** by 0.09/0.07 AUROC (0.08/0.06 AUPRC) and **Global-Rubric** by 0.04/0.06 AUROC (0.04/0.07 AUPRC).

Local versus global rubrics: interpretation helps most when labels are scarce. **Local-Rubric** is strongest at $n = 10$ (0.701 AUROC, 0.382 AUPRC), while **Global-Rubric** nearly closes the gap by $n = \text{All}$ and slightly exceeds **Local-Rubric** in overall AUPRC (0.459 vs. 0.452). This pattern suggests that the interpretive language in **Local-Rubric** is most useful when the downstream classifier has few labels. **Local-Rubric** outputs often translate raw findings into task-conditioned clinical and statistical language, whereas **Global-Rubric** extracts similar evidence into standardized fields without explicit risk interpretation (Figure 6, left and right). Thus, **Local-Rubric** appears to provide a useful inductive bias/prior in the language representation at small n ; as labels accumulate, the downstream classifier can increasingly learn from the structured evidence itself, and the advantage of explicit interpretation diminishes.

Ablations separate the effects of interpretation and standardization. **Local-Rubric-NoInterp** preserves the patient-level evidence and missingness cues from **Local-Rubric** while removing explicit risk-stratification language. Its drop at small n , followed by convergence with **Local-Rubric** at $n = \text{All}$, is consistent with interpretive LLM-generated language acting as an inductive bias when labels are scarce. **Local-Rubric-Basic** removes both this interpretive layer and some of the richer missingness/context cues. Compared with **Global-Rubric**, which also avoids explicit risk interpretation but imposes a standardized template, the two are comparable at $n = 10$, while **Global-Rubric** pulls ahead at $n = \text{All}$. This suggests that global standardization becomes increasingly valuable once the downstream classifier has enough labels to exploit the structured representation.

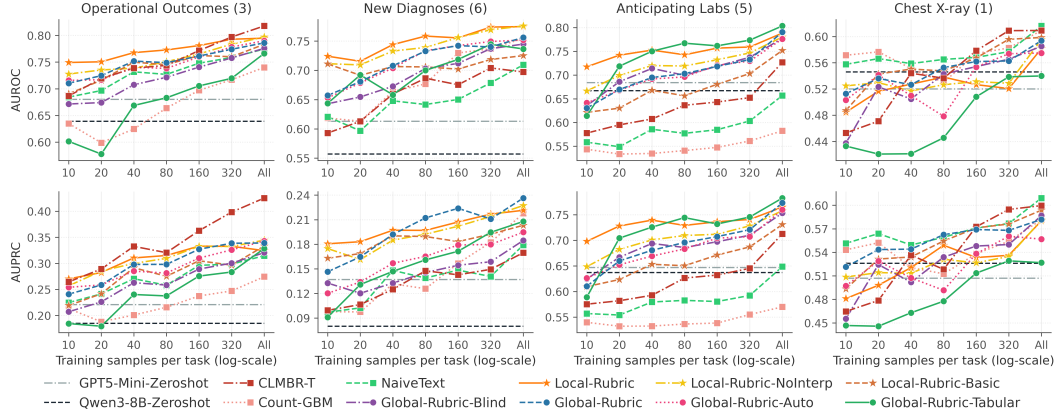


Figure 7: Average performance by task group. For confidence bands, see Appendix E.3.

Global rubric variants: cost of automation. **Global-Rubric-Auto**, which replaces per-example LLM calls with a deterministic parser, tracks **Global-Rubric** closely on AUROC across sample sizes and gives up about 0.02 AUPRC at $n=\text{All}$ (0.437 vs 0.459), recovering most of the gain over baselines at near-zero marginal LLM cost. **Global-Rubric-Tabular** lags visibly at small n , which we attribute to XGBoost’s inability to leverage a prior, unlike the text-embedding models used in other methods. As expected, it nearly catches up at $n=\text{All}$, and on lab tasks it is in fact the single strongest method overall. Finally, **Global-Rubric-Blind** generates the rubric from the task description alone. It continues to outperform the baselines across sample sizes but remains inferior to **Global-Rubric**, showing the value of letting the LLM examine some data first (Panel A in Figure 3).

Operational outcomes. **CLMBR-T** is the strongest here at $n=\text{All}$ (0.818 AUROC, 0.425 AUPRC), edging out **Local-Rubric** (0.802, 0.341) and **Global-Rubric** (0.786, 0.339), with a more pronounced lead in AUPRC. We attribute this to alignment between **CLMBR-T**’s next-code pretraining and operational targets. For instance, every visit in the pretraining corpus contributes an implicit “label” for the ICU admission task via next-code prediction, which is not the case for other tasks such as lab results. Even so, in the small-sample regime our rubrics overtake **CLMBR-T** in AUROC (**Local-Rubric** 0.750, **Global-Rubric** 0.710 at $n=10$, vs **CLMBR-T** 0.687) and match it in AUPRC.

Lab results. Rubric methods deliver large gains here. At $n=\text{All}$, **Global-Rubric-Tabular** is the best method, and strong small-sample performance of **Local-Rubric** prevails. Rubrics align well with this type of tasks, where the predictive signal lives in a compact, recency-aware set of measurements (recent labs, trends, contributing medications). The lab rubrics (Appendix G) expose those directly, whereas raw serializations leave them scattered across visit narratives, buried in noise.

New diagnoses and chest X-ray. Rubrics dominate the new-diagnosis group at small n (**Local-Rubric** 0.713/0.170 AUROC/AUPRC vs **Count-GBM** 0.619/0.098 at $n=10$), roughly 1.5–2 $\times$ above baseline AUPRC and especially strong on rare-event tasks (Tables 13 and 14). Chest X-ray is the noisiest task in EHRSHOT. The binary label collapses an originally 14-way radiology label, and many patients have their last documented visit more than a month before the prediction time. All methods cluster near 0.60 AUROC at $n=\text{All}$, consistent with limited learnable signal in this label given the available context, rather than a method-specific failure. We cannot, however, fully rule out room for method-specific improvement.

Robustness across text-embedding backbones. The rubric-over-**NaiveText** ordering is preserved with Mistral-7B, LLaMA-3-8B (LLM2Vec), and OpenAI’s text-embedding-3-large (Appendix E.2), in both AUROC and AUPRC.

6 Qualitative analysis of a global rubric for hypertension onset prediction

We examine the learned global rubric for the new hypertension diagnosis prediction task. An excerpt is shown in Figure 8, with the full rubric in Appendix G.1.

Temporal standardization and BP normalization. The rubric first imposes structure on noisy EHR data before extracting features. In the preparation stage (Figure 8, A), it defines explicit time

A. Preparation (before extracting)

- Define time windows: *Very recent*: last 30 days, *Recent*: 31-180 days, *Baseline/remote*: >180 days
- Standardize units: *Blood pressure*: mmHg (systolic/diastolic), *Weight*: kg or oz → convert to kg [...]

Step 2 - Blood pressure (BP) data extraction and normalization

- Extract systolic/diastolic BP values with timestamps and context (office, inpatient, ED, home, ambulatory, perioperative).
- For each time window (very recent, recent, baseline): compute count, mean, median, SD, min, max; flag highest recent BP
- Compute simple trend metrics (e.g., recent slope; BP variability via SD).
- Categorize BP per ACC/AHA categories using aggregated recent values: Normal (<120/<80), Elevated (120-129/<80) [...]

Step 9 - Synthesis per domain (structured fields and scoring)

- For each domain, record presence, supporting data, recency, and confidence (High/Moderate/Low).
- *Domain A* - BP phenotype: last BP (date/context), mean recent BP (last 30d; 31-180d), BP category, variability flag, ambulatory BP.
- *Domain B* - Metabolic / vascular risk: Diabetes (Y/N) - last A1c (% and date), BMI and obesity category, Hyperlipidemia (Y/N) - LDL value and date, Smoking (current/former/never). Create a simple domain scorecard: number of High/Moderate/Minor risk features [...]

Figure 8: Key excerpts from learned hypertension global rubric instructions.

windows: very recent (≤ 30 days), recent (31–180 days), and baseline/remote (> 180 days), and standardizes units and formats, including blood pressure (BP) in mmHg and weight in kg. This is important for hypertension prediction because recency, persistence, and measurement comparability are central to distinguishing sustained hypertension risk from isolated or context-specific elevations.

Clinically grounded BP feature construction. Step 2 converts raw BP readings into structured longitudinal features. The rubric first extracts systolic/diastolic values with timestamps and clinical context. Within each temporal window, it computes summary statistics such as mean and maximum. It also derives simple trend and variability measures such as recent slope and standard deviation (SD). Finally, it maps aggregated recent BP values to ACC/AHA categories, converting irregular raw measurements into clinically interpretable BP phenotype features [60].

Domain-level synthesis. Step 9 further compresses the extracted evidence into domain-level summaries and a scorecard. For each domain, the rubric records presence, supporting data, recency, and confidence. Domain A summarizes BP phenotype, including last BP, recent means, BP category, variability, and ambulatory BP; Domain B captures metabolic and vascular risk factors such as diabetes, BMI/obesity, hyperlipidemia, LDL, and smoking. The resulting counts of high-, moderate-, and minor-risk features provide compact task-specific signals that can be used to effectively predict the cumulative hypertension risk across BP patterns and comorbidities.

More broadly, this highlights the value of global rubrics as a bridge between raw, heterogeneous inputs and downstream prediction. By making task-relevant evidence explicit and structured, rubrics provide models with more interpretable, standardized, and prediction-aligned representations. As a complementary case study, in Appendix D we examine the *learned tabular features* produced by [Global-Rubric-Tabular](#) (Figure 3, Panel F) for the hyponatremia lab task and show that they closely track the standard clinical decision tree for evaluating hyponatremia.

7 Concluding remarks and limitations

We proposed *rubric representation learning*, where LLMs transform naive text-serializations into task-aligned representations before downstream training. Across 15 EHRSHOT tasks, rubric methods substantially outperformed baselines, particularly for small n . [Local-Rubric](#) was strongest at small n , suggesting that LLMs’ pretrained knowledge can act as a major contributor to downstream performance. Its performance was matched by [Global-Rubric](#) at large n , demonstrating the advantages of having a standardized input representation. Our ablations decompose the rubric advantage into two complementary levers, an LLM-injected statistical prior that drives performance at small n and a standardized template that adds representational value at scale. We further showed that automated global-rubric variants remain competitive while drastically reducing inference cost.

Limitations and future work. Our evaluation is restricted to a single benchmark and does not include richer modalities such as clinical notes or images. Currently, rubric synthesis is bounded by context length (40 patients per global rubric), and iterative refinement using additional examples, failure cases, or expert feedback is a natural next step. Further, we report a single global rubric per task. Assessing the sensitivity of downstream performance to the cohort sampled for rubric synthesis is an important but costly direction for future work. Finally, clinical deployment should require expert review, subgroup evaluation, privacy safeguards, and monitoring for errors or distribution shift.

References

- [1] L. A. Agrawal, S. Tan, D. Soylu, N. Ziemis, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026.
- [2] M. Agrawal, S. Hegselmann, H. Lang, Y. Kim, and D. Sontag. Large language models are few-shot clinical information extractors. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 1998–2022, 2022.
- [3] Y. Akhauri, B. Lewandowski, C.-H. Lin, A. N. Reyes, G. C. Forbes, A. Wongpanich, B. Yang, M. S. Abdelfattah, S. Perel, and X. Song. Performance prediction for large systems via text-to-text regression. *arXiv preprint arXiv:2506.21718*, 2025.
- [4] S. Bedi, H. Cui, M. Fuentes, A. Unell, M. Wornow, J. M. Banda, N. Kotecha, T. Keyes, Y. Mai, M. Oez, et al. Holistic evaluation of large language models for medical tasks with medhelm. *Nature Medicine*, pages 1–9, 2026.
- [5] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy. LLM2Vec: Large language models are secretly powerful text encoders. *arXiv preprint arXiv:2404.05961*, 2024. URL <https://arxiv.org/abs/2404.05961>.
- [6] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [7] K. Choi, C. Cundy, S. Srivastava, and S. Ermon. Lmpriors: Pre-trained language models as task-specific priors. *arXiv preprint arXiv:2210.12530*, 2022.
- [8] I. Demirel, K. Thakkar, B. Elizalde, S. Y. Ren, and J. Narain. Using llms for late multimodal sensor fusion for activity recognition. In *NeurIPS 2025 Workshop on Learning from Time Series for Health*, 2025.
- [9] S. L. Fleming, A. Lozano, W. J. Haberkorn, J. A. Jindal, E. Reis, R. Thapa, L. Blankemeier, J. Z. Jenkins, E. Steinberg, A. Nayak, et al. Medalign: A clinician-generated dataset for instruction following with electronic medical records. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 22021–22030, 2024.
- [10] P. Garcia, S. P. Ma, S. Shah, M. Smith, Y. Jeong, A. Devon-Sand, M. Tai-Seale, K. Takazawa, D. Clutter, K. Vogt, et al. Artificial intelligence-generated draft replies to patient inbox messages. *JAMA Network Open*, 7(3):e243201, 2024.
- [11] E. Goh, R. J. Gallo, E. Strong, Y. Weng, H. Kerman, J. A. Freed, J. A. Cool, Z. Kanjee, K. P. Lane, A. S. Parsons, et al. Gpt-4 assistance for improvement of physician performance on patient care tasks: a randomized controlled trial. *Nature Medicine*, 31(4):1233–1238, 2025.
- [12] V. Gulshan, L. Peng, M. Coram, M. C. Stumpe, D. Wu, A. Narayanaswamy, S. Venugopalan, K. Widner, T. Madams, J. Cuadros, R. Kim, R. Raman, P. C. Nelson, J. L. Mega, and D. R. Webster. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *JAMA*, 316(22):2402–2410, 2016.
- [13] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang. DS-agent: Automated data science by empowering large language models with case-based reasoning. In *International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 16813–16848, 2024.
- [14] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag. Tabllm: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 5549–5581. PMLR, 2023.
- [15] S. Hegselmann, G. von Arnim, T. Rheude, N. Kronenberg, D. Sontag, G. Hindricks, R. Eils, and B. Wild. Large language models are powerful electronic health record encoders. *arXiv preprint arXiv:2502.17403*, 2025.

- [16] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, C. Zhang, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, L. Zhang, M. Yang, M. Zhuge, T. Guo, T. Zhou, W. Tao, R. Tang, X. Lu, X. Zheng, X. Liang, Y. Fei, Y. Cheng, Y. Ni, Z. Gou, Z. Xu, Y. Luo, and C. Wu. Data interpreter: An LLM agent for data science. In *Association for Computational Linguistics (ACL)*, pages 19796–19821, 2025.
- [17] D. P. Jeong, Z. C. Lipton, and P. K. Ravikumar. LLM-select: Feature selection with large language models. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.
- [18] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023. URL <https://arxiv.org/abs/2310.06825>.
- [19] Y. Jiang, K. C. Black, G. Geng, D. Park, J. Zou, A. Y. Ng, and J. H. Chen. Medagentbench: a virtual ehr environment to benchmark medical llm agents. *Nejm Ai*, 2(9), 2025.
- [20] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021.
- [21] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [22] J. Kim, A. Podlasek, K. Shidara, F. Liu, A. Alaa, and D. Bernardo. Limitations of large language models in clinical problem-solving arising from inflexible reasoning. *Scientific reports*, 15(1): 39426, 2025.
- [23] J. Kim, C. Squires, and P. Ravikumar. Knowledge-enriched machine learning for tabular data. In *International Conference on Neuro-symbolic Systems*, pages 260–292, 2025.
- [24] M. Kirchler, M. Ferro, V. Lorenzini, R. P. van de Water, F. G. A. 3, C. Lippert, and A. Ganna. Large language models improve transferability of electronic health record-based predictions across countries and coding systems. *npj Digital Medicine*, 2026.
- [25] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman, L. J. V. Miranda, A. Liu, N. Dziri, S. Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- [26] S. Lessmann, B. Baesens, H.-V. Seow, and L. Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring. *European Journal of Operational Research*, 247(1):124–136, 2015.
- [27] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.
- [28] Y. Liao, C. Wu, J. Liu, S. Jiang, P. Qiu, H. Wang, Y. Yue, S. Zhen, J. Wang, Q. Fan, et al. Ehr-r1: A reasoning-enhanced foundational language model for electronic health record analysis. *arXiv preprint arXiv:2510.25628*, 2025.
- [29] J. Lin, Z. Wu, and J. Sun. Training llms for ehr-based reasoning tasks via reinforcement learning. *arXiv preprint arXiv:2505.24105*, 2025.
- [30] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [31] D. McDuff, M. Schaekermann, T. Tu, A. Palepu, A. Wang, J. Garrison, K. Singhal, Y. Sharma, S. Azizi, K. Kulkarni, et al. Towards accurate differential diagnosis with large language models. *Nature*, 642(8067):451–457, 2025.
- [32] N. Mehandru, B. Y. Miao, E. R. Almaraz, M. Sushil, A. J. Butte, and A. Alaa. Evaluating large language models as agents in the clinic. *NPJ digital medicine*, 7(1):84, 2024.
- [33] Meta AI. The Llama 3 herd of models. Technical report, Meta, 2024. URL <https://arxiv.org/abs/2407.21783>.

- [34] J. Nam, J. Yoon, J. Chen, R. Sinha, J. Shin, and T. Pfister. DS-STAR: Data science agent for solving diverse tasks across heterogeneous formats and open-ended queries. *arXiv preprint arXiv:2509.21825*, 2025.
- [35] OpenAI. New embedding models and API updates. <https://openai.com/blog/new-embedding-models-and-api-updates>, 2024. Accessed: 2025-06-15.
- [36] OpenAI. Gpt-5 system card. <https://cdn.openai.com/gpt-5-system-card.pdf>, Aug. 2025.
- [37] J. W. O’Sullivan, A. Palepu, K. Saab, W.-H. Weng, D. K. Amponsah, E. Cheng, Y. Cheng, E. Chu, Y. Desai, A. Elezaby, et al. A large language model for complex cardiology care. *Nature Medicine*, pages 1–8, 2026.
- [38] A. Palepu, V. Dhillon, P. Niravath, W.-H. Weng, P. Prasad, K. Saab, R. Tanno, Y. Cheng, H. Mai, E. Burns, et al. Exploring large language models for specialist-level oncology care. *NEJM AI*, 2(11):A1cs2500025, 2025.
- [39] P. Petridis, G. Margaritis, V. Stoumpou, and D. Bertsimas. Holistic ai in medicine; improved performance and explainability. *npj Digital Medicine*, 2026.
- [40] Qwen3Team. Qwen3 technical report, 2025.
- [41] P. Rajpurkar, J. Irvin, K. Zhu, B. Yang, H. Mehta, T. Duan, D. Ding, A. Bagul, C. Langlotz, K. Shpanskaya, M. P. Lungren, and A. Y. Ng. Chexnet: Radiologist-level pneumonia detection on chest x-rays with deep learning. *arXiv preprint arXiv:1711.05225*, 2017.
- [42] S. Ravuri, K. Lenc, M. Willson, D. Kangin, R. Lam, P. Mirowski, S. Fitzpatrick, M. Athanasiadou, S. Kashem, S. Madge, R. Prudden, A. Mandhane, A. Clark, A. Brock, K. Simonyan, R. Hadsell, N. Robinson, E. Clancy, A. Arribas, S. Mohamed, and N. Kalchbrenner. Skilful precipitation nowcasting using deep generative models of radar. *Nature*, 597:672–677, 2021.
- [43] P. Renc, Y. Jia, A. E. Samir, J. Was, Q. Li, D. W. Bates, and A. Sitek. Zero shot health trajectory prediction using transformer. *NPJ digital medicine*, 7(1):256, 2024.
- [44] S. Sandmann, S. Hegselmann, M. Fajarski, L. Bickmann, B. Wild, R. Eils, and J. Varghese. Benchmark evaluation of deepseek large language models in clinical decision-making. *Nature medicine*, 31(8):2546–2549, 2025.
- [45] R. Shao, A. Asai, S. Z. Shen, H. Ivison, V. Kishore, J. Zhuo, X. Zhao, M. Park, S. G. Finlayson, D. Sontag, et al. Dr tulur: Reinforcement learning with evolving rubrics for deep research. *arXiv preprint arXiv:2511.19399*, 2025.
- [46] W. Shi, R. Xu, Y. Zhuang, Y. Yu, J. Zhang, H. Wu, Y. Zhu, J. C. Ho, C. Yang, and M. D. Wang. Ehragent: Code empowers large language models for few-shot complex tabular reasoning on electronic health records. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 22315–22339, 2024.
- [47] C. Si, Z. Yang, Y. Choi, E. Candès, D. Yang, and T. Hashimoto. Towards execution-grounded automated ai research. *arXiv preprint arXiv:2601.14525*, 2026.
- [48] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620(7972):172–180, 2023.
- [49] K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, M. Amin, L. Hou, K. Clark, S. R. Pfohl, H. Cole-Lewis, et al. Toward expert-level medical question answering with large language models. *Nature medicine*, 31(3):943–950, 2025.
- [50] X. Song, O. Li, C. Lee, B. Yang, D. Peng, S. Perel, and Y. Chen. Omnipred: Language models as universal regressors. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- [51] G. Spasovski, R. Vanholder, B. Allolio, D. Annane, S. Ball, D. Bichet, G. Decaux, W. Fenske, E. J. Hoorn, C. Ichai, et al. Clinical practice guideline on diagnosis and treatment of hyponatraemia. *Nephrology Dialysis Transplantation*, 29(suppl_2):i1–i39, 2014.

- [52] E. Steinberg, J. A. Fries, Y. Xu, and N. Shah. MOTOR: A time-to-event foundation model for structured medical records. In *International Conference on Learning Representations (ICLR)*, 2024.
- [53] T. Tu, M. Schaekermann, A. Palepu, K. Saab, J. Freyberg, R. Tanno, A. Wang, B. Li, M. Amin, Y. Cheng, et al. Towards conversational diagnostic artificial intelligence. *Nature*, 642(8067): 442–450, 2025.
- [54] O. Unlu, J. Shin, C. J. Mailly, M. F. Oates, M. R. Tucci, M. Varugheese, K. Waghlikar, F. Wang, B. M. Scirica, A. J. Blood, et al. Retrieval-augmented generation-enabled gpt-4 for clinical trial screening. *NEJM AI*, 1(7), 2024.
- [55] A. Upadhyay, B. L. Jaber, and N. E. Madias. Incidence and prevalence of hyponatremia. *The American journal of medicine*, 119(7):S30–S35, 2006.
- [56] J. G. Verbalis, S. R. Goldsmith, A. Greenberg, C. Korzelius, R. W. Schrier, R. H. Sterns, and C. J. Thompson. Diagnosis, evaluation, and treatment of hyponatremia: expert panel recommendations. *The American journal of medicine*, 126(10):S1–S42, 2013.
- [57] P. Wan, Z. Huang, W. Tang, Y. Nie, D. Pei, S. Deng, J. Chen, Y. Zhou, H. Duan, Q. Chen, et al. Outpatient reception via collaboration between nurses and a large language model: a randomized controlled trial. *Nature Medicine*, 30(10):2878–2885, 2024.
- [58] S. Waxler, P. Blazek, D. White, D. Sneider, K. Chung, M. Nagarathnam, P. Williams, H. Voeller, K. Wong, M. Swanhorst, et al. Generative medical event models improve with scale. *arXiv preprint arXiv:2508.12104*, 2025.
- [59] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
- [60] P. K. Whelton, R. M. Carey, W. S. Aronow, D. E. Casey, K. J. Collins, C. Dennison Himmelfarb, S. M. DePalma, S. Gidding, K. A. Jamerson, D. W. Jones, et al. 2017 acc/aha/aapa/abc/acpm/ags/apha/ash/aspc/nma/pcna guideline for the prevention, detection, evaluation, and management of high blood pressure in adults: a report of the american college of cardiology/american heart association task force on clinical practice guidelines. *Journal of the American College of Cardiology*, 71(19):e127–e248, 2018.
- [61] M. Wornow, R. Thapa, E. Steinberg, J. Fries, and N. Shah. EHRSHOT: An ehr benchmark for few-shot evaluation of foundation models. *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 36:67125–67137, 2023.
- [62] A. Yalamanchili, B. Sengupta, J. Song, S. Lim, T. O. Thomas, B. B. Mittal, M. E. Abazeed, and P. T. Teo. Quality of large language model responses to radiation oncology patient care questions. *JAMA network open*, 7(4):e244630, 2024.
- [63] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- [64] K. Yoon, L. Mao, C. Chong, T. J. Schwedt, C.-C. Chiang, and J. Li. Paregta: An llm-based ehr data encoding approach to capture temporal information. *arXiv preprint arXiv:2602.19661*, 2026.
- [65] M. Zayyan. Objective structured clinical examination: the assessment of choice. *Oman medical journal*, 26(4):219, 2011.
- [66] E. Zhang, R. Goto, N. Sagan, J. Mutter, N. Phillips, A. Alizadeh, K. Lee, J. Blanchet, M. Pilanci, and R. Tibshirani. Llm-lasso: A robust framework for domain-informed feature selection and regularization. *arXiv preprint arXiv:2502.10648*, 2025.
- [67] S. Zhang, J. Fan, M. Fan, G. Li, and X. Du. Deepanalyze: Agentic large language models for autonomous data science. *arXiv preprint arXiv:2510.16872*, 2025.

- [68] S. Zhang, Q. Liu, G. Qin, T. Naumann, and H. Poon. Med-rlvr: Emerging medical reasoning from a 3b base model via reinforcement learning. *arXiv preprint arXiv:2502.19655*, 2025.
- [69] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.

A Extended related work

Data science agents. A complementary line of work studies “data science (DS) agents”. Data Interpreter targets end-to-end problem solving, using code generation, execution, and revision to complete data analysis and mathematical tasks [16]. DS-Agent focuses on automating model development workflows such as task understanding, model selection, and training [13]. DeepAnalyze and DS-STAR push further toward autonomous data science over heterogeneous files, with an emphasis on multi-step data wrangling, open-ended querying, code execution, and report generation [34, 67]. This literature is closely related to our work in using LLMs as an interface to heterogeneous data. Our focus, however, is narrower and more controlled. Rather than asking LLM agents to plan and execute broad analyses, we study how LLMs can support input representation design with complex data for specific downstream tasks. This lets us isolate the role of representation choice and demonstrate its effect as a first-order driver of downstream statistical performance.

Medical QA benchmarks. Recent work demonstrated that LLMs encode substantial clinical knowledge. Singhal et al. [48] introduced Med-PaLM, a closed-source medical LLM from Google. They evaluated it on MultiMedQA, a benchmark combining six medical question answering datasets, and showed that instruction-tuned models could surpass prior state-of-the-art. The follow-up Med-PaLM 2 pushed further, achieving performance competitive with expert physicians [49]. Sandmann et al. [44] evaluated open-source DeepSeek models on clinical decision support tasks using 125 patient cases. They found that the open-source frontier models perform equally well, if not better, than proprietary models. Zhang et al. [68] showed that even a 3B-parameter model can develop some medical reasoning skills when trained via reinforcement learning with verifiable rewards (RLVR, [25]) on the MedQA benchmark [20].

Moving beyond medical QA benchmarks. While these results are impressive, high scores on medical QA do not translate to clinical impact. Mehandru et al. [32] proposed AI-based Standardized Clinical Examination (AI-SCE), modeled after the Objective Structured Clinical Examination (OSCE, [65]) used in medical training, to evaluate LLMs as agents in realistic, multi-step clinical scenarios rather than static question-answering benchmarks. Kim et al. [22] identified that LLMs can exhibit inflexible reasoning in clinical problem-solving and struggle with unexpected *long-tail* situations. Bedi et al. [4] introduced MedHELM, a holistic evaluation framework that organizes medical tasks into a taxonomy spanning clinical decision support, note generation, patient communication, research, and administration. Their analyses revealed that LLMs perform more variably on realistic clinical tasks than on standardized exams. Jiang et al. [19] developed MedAgentBench, a virtual EHR environment specifically designed to benchmark LLM agents on multi-step clinical tasks.

LLMs in the clinic. Recent work probes how LLMs can be deployed to support clinicians and patients. Garcia et al. [10] demonstrated that AI-generated draft replies to patient messages can reduce physician burden without sacrificing quality. Yalamanchili et al. [62] evaluated the quality of LLM responses to radiation oncology patient queries. Unlu et al. [54] showed that retrieval-augmented GPT-4 (RAG, [27]) can assist with clinical trial screening. Randomized trials have also begun to probe whether LLMs can reliably improve clinicians’ performance. Goh et al. [11] conducted a trial and found that GPT-4 assistance improved physician performance, while Wan et al. [57] showed that nurse-LLM collaboration for outpatient reception resulted in “increased satisfaction among both patients and nurses”.

Conversational diagnostic models and specialist systems. Google’s Articulate Medical Intelligence Explorer (AMIE) represents a sustained research program on LLMs in healthcare. Tu et al. [53] introduced AMIE, a conversational and diagnostic AI system, trained via self-play in a simulated environment to conduct multi-turn clinical conversations. They showed that it outperformed primary

care physicians on 30 of 32 evaluation axes in a randomized, blinded study using standardized patient actors. McDuff et al. [31] evaluated AMIE’s capacity for differential diagnosis, demonstrating that it generates diagnostic lists that exceed GPT-4’s quality and improve clinicians’ diagnostic accuracy when used as an assistive tool. Subsequent work extended AMIE to specialist domains. Palepu et al. [38] evaluated its performance in oncology care, and O’Sullivan et al. [37] reported results for complex cardiology cases.

LLMs with EHR data. There is a growing literature developing medical foundation models pretrained on large EHR or claims data for risk prediction and trajectory modeling [43, 52, 58]. Recent work shows general-purpose LLMs can match domain-specific models on clinical tasks [15], which we reproduce and strengthen with our LLM-derived rubrics.

Agrawal et al. [2] showed that LLMs are effective clinical information extractors, pulling structured data from unstructured clinical text with few examples. Fleming et al. [9] released MedAlign, a clinician-generated dataset for instruction following, targeting realistic EHR-grounded tasks. Shi et al. [46] proposed EHRAgent, an LLM agent that generates and executes code to answer clinician queries. Lin et al. [29] combined supervised fine-tuning with reinforcement learning, demonstrating gains across medical calculation, patient-trial matching, and disease diagnosis in EHRSHOT benchmark. Liao et al. [28] developed EHR-R1, a reasoning-enhanced model for EHR analysis using reinforcement learning. Kirchler et al. [24] demonstrated that LLM-based clinical prediction models can have improved cross-country and system transferability. Yoon et al. [64] proposed an encoding approach for EHR data using LLMs to better emphasize temporal information.

Connections to AI for science and research. There is a growing literature on using LLMs to automate parts of the scientific process. Systems such as the AI Scientist [30, 63] generate hypotheses and iteratively design experiments. Other work emphasizes execution-grounded research pipelines, where LLM-generated plans are validated through execution and feedback [47]. Shao et al. [45] explore evolving rubrics to guide multi-step research. These directions are conceptually related to our setting: LLMs are used to generate intermediate representations and pipelines for complex tasks and datasets, that are subsequently executed and evaluated for downstream objectives.

B EHRSHOT subsampling procedure and dataset statistics

Table 1: Number of samples (positive cases) per task and split. All splits for Lab results and Chest X-ray tasks are subsampled from the original dataset. Validation splits are subsampled for all tasks.

Category	Task	Train	Val	Test
Operational Outcomes (3)	ICU transfer	2402 (113)	100 (50)	2037 (85)
	Length of stay >7 days	2569 (681)	100 (50)	2195 (552)
	30-day readmission	2608 (370)	100 (50)	2189 (260)
Assignment of New Diagnosis (6)	Hypertension	1259 (182)	100 (50)	1258 (159)
	Hyperlipidemia	1684 (205)	100 (50)	1317 (172)
	Pancreatic cancer	2576 (155)	100 (50)	2220 (56)
	Celiac disease	2623 (62)	22 (11)	2222 (21)
	Lupus	2570 (104)	66 (33)	2243 (20)
	Acute MI	2534 (175)	100 (50)	2127 (144)
Anticipating Labs (5)	Thrombocytopenia	2000 (1000)	100 (50)	2000 (1000)
	Hyperkalemia	2000 (1000)	100 (50)	1896 (948)
	Hypoglycemia	2000 (1000)	100 (50)	1566 (783)
	Hyponatremia	2000 (1000)	100 (50)	2000 (1000)
	Anemia	2000 (1000)	100 (50)	2000 (1000)
Chest X-ray Findings (1)	Chest X-ray abnormality	2000 (1000)	100 (50)	2000 (1000)

We evaluate on the 15 binary prediction tasks from EHRSHOT [61] listed in Section 3. Several of our methods invoke an LLM call per example, so we subsample some splits for budget reasons. Final per-split sample counts are reported in Table 1.

Let n_+ denote the number of positive examples in the corresponding original split. Validation sets are label-balanced with up to $\min(50, n_+)$ positives and the same number of negatives, so most validation sets contain 50/50. For operational and diagnosis tasks we retain the original EHRSHOT training and test splits, which are moderate in size. For lab and chest X-ray tasks the original splits are substantially larger, and we subsample training and test splits to up to $\min(1000, n_+)$ positives and the same number of negatives, yielding balanced subsets of up to 2,000 examples per split; when fewer positives are available, negatives are matched to the available positives.

C Reproduction and compute costs

We summarize the practical resource requirements for reproducing our experiments on the subsampled EHRSHOT splits described above (Section B, Table 1).

NaiveText truncation. Following Hegselmann et al. [15], we clip [NaiveText](#) serializations at 8,192 tokens (Qwen3-8B tokenizer).

LLM API costs. Methods with a per-example LLM call—[Global-Rubric](#), [Global-Rubric-Blind](#), [Local-Rubric](#), and [Local-Rubric-Basic](#)—each cost approximately \$50 per task in GPT5-Mini API usage on our subsampled splits, totaling roughly \$3,000 across 4 methods and 15 tasks. Global-rubric synthesis and the one-time parser/tabularization scripts used by [Global-Rubric-Auto](#) and [Global-Rubric-Tabular](#) (Figure 3, Panels E–F) add a small one-off cost; once those scripts are produced, applying them to new examples at deployment time is essentially free.

Embedding compute. Text embeddings (Qwen3-Embedding-8B for all main results, plus the open-weights backbones used in Section E.2) are computed on a node with 4 NVIDIA A100 GPUs. Embedding the full dataset for any given representation method ([NaiveText](#) or any rubric variant) takes a few hours per method.

Downstream training. The downstream classifiers (logistic regression on frozen embeddings; LightGBM for [Count-GBM](#); XGBoost for [Global-Rubric-Tabular](#)) are lightweight and run on commodity CPU hardware in seconds to minutes per task; their cost is negligible compared to the embedding and LLM-call steps above.

D Case study: learned tabular features for prediction of hyponatremia

As a complement to the qualitative analysis in Section 6, we examine the learned tabular features (Figure 3, Panel F) for prediction of hyponatremia abnormality. Across the 15 tasks, the auto-generated rubric feature schemas range from 147 to 450 features per task and cluster around 200–250 features. They are predominantly binary (72%), followed by numeric (19%) and categorical (9%). The high binary share reflects pervasive one-hot encoding of categoricals and the inclusion of a `_missing` indicator for nearly every field. Numeric features capture lab values, vitals, and counts.

We focus on learned tabular features for the hyponatremia lab task, for which the full global rubric is given in Appendix G.2. We make several observations. The feature structure closely mirrors the diagnostic decision tree used clinically when evaluating hyponatremia. A first step in clinical reasoning is determining whether apparent hyponatremia is physiologic or artificially low due to hyperglycemia or other osmotic effects; accordingly, the rubric extracts recent glucose measurements (`Glucose-Last3`) and serum osmolality values, while the tabular features include indicators reflecting level of glucose in the blood, *e.g.* `glucose-type-blood-present`. If true hypotonic hyponatremia is present, clinicians next evaluate urine osmolality and urine sodium to distinguish between states of antidiuretic hormone (ADH) activity and renal sodium handling, which helps identify etiologies such as SIADH or hypovolemia [51]. Consistent with this framework, the rubric extracts urine sodium, urine osmolality, and conditions associated with SIADH (*e.g.*, pulmonary infections, CNS disorders, malignancy). The resulting features include indicators that reflect such conditions, *e.g.*, `acute-cond-Pulmonary infection / pneumonia / pulmonary disease` and `acute-cond-any`. Thus, the learned features directly operationalize the same diagnostic flow in clinical practice.

A second group of features captures baseline risk factors and comorbidities that predispose patients to hyponatremia. The rubric explicitly extracts conditions such as chronic kidney disease (CKD), dialysis history, malignancy, and medications known to induce hyponatremia (*e.g.*, thiazide diuretics). These signals appear directly in the tabularized representation through features such as `dialysis-history-Yes`, `procedure-Hemodialysis`, and `med-class-count-thiazide-diuretic`. These variables correspond to well-known clinical risk factors for hyponatremia, including impaired renal free-water handling and medication-induced sodium loss [56].

Finally, the most predictive signals arise from the acuity and trajectory of prior sodium measurements. The rubric explicitly extracts the three most recent sodium values and the lowest sodium in the prior 90 days, along with contextual metadata such as the setting of the measurement (*e.g.*, inpatient vs outpatient). Correspondingly, the largest-magnitude coefficients in the tabular feature set correspond to prior sodium measurements, including `serum-na-recent-le-134` and `prior-documented-hyponatremia-Yes`. Clinically, this is expected, as patients with a history of chronic or recurrent hyponatremia (*e.g.*, due to heart failure or cirrhosis) are substantially more likely to have abnormal sodium levels on subsequent laboratory testing [55].

This example illustrates how rubric representations surface task-relevant information that would otherwise be buried in a long, heterogeneous text-serialization of the patient record. In the naive text format, prior sodium measurements appear scattered across multiple visits and lab panels, interleaved with unrelated clinical events. The rubric reorganizes this information into a compact set of fields that explicitly capture the recent trajectory of the lab. In doing so, it converts diffuse signals in language space into structured features that a simple downstream model can use efficiently.

E Additional quantitative results

E.1 Average numerical results for $n = 10$ and $n = \text{All}$ regimes

Table 2: AUROC and AUPRC across all methods for $n = 10$ and $n = \text{All}$. **Green:** best in entire column. **Blue:** best within a method across backbones. **Oper.:** Operational Outcomes, **New Dx:** Assignment of New Diagnosis, **Labs:** Anticipating Lab Results, **CXR:** Chest X-ray Findings

		(a) AUROC				
		Overall (15)	Oper. (3)	New Dx (6)	Labs (5)	CXR (1)
$n = 0$	Qwen3-8B-Zeroshot	.610.601-.618	.639.616-.660	.557.541-.575	.667.658-.677	.546.519-.573
	GPT5-Mini-Zeroshot	.644.635-.653	.680.658-.700	.613.594-.632	.684.675-.694	.520.498-.541
$n = 10$	Count-GBM	.594.579-.609	.635.610-.658	.619.583-.652	.544.532-.555	.571.541-.602
	CLMBR-T	.597.582-.613	.687.662-.709	.593.559-.629	.578.567-.588	.453.421-.484
	NaiveText	.609.595-.621	.684.664-.705	.621.591-.649	.559.547-.570	.558.533-.584
	Local-Rubric	.701.687-.715	.750.730-.768	.713.681-.743	.694.684-.705	.517.492-.543
	Local-Rubric-Basic	.662.648-.675	.691.670-.711	.711.682-.741	.622.610-.632	.487.457-.519
	Local-Rubric-NoInterp	.687.673-.701	.728.708-.746	.711.680-.741	.666.656-.677	.525.491-.556
	Global-Rubric-Blind	.632.619-.646	.671.651-.691	.643.614-.672	.636.624-.647	.437.413-.461
	Global-Rubric	.649.635-.662	.710.690-.732	.657.627-.688	.630.619-.641	.513.489-.538
	Global-Rubric-Auto	.652.639-.666	.716.695-.736	.653.625-.683	.641.630-.653	.503.470-.534
$n = \text{All}$	Global-Rubric-Tabular	.616.605-.627	.601.575-.628	.644.619-.667	.614.603-.625	.500.500-.500
	Count-GBM	.685.670-.699	.740.717-.761	.755.722-.786	.583.572-.594	.609.576-.642
	CLMBR-T	.725.711-.739	.818.799-.836	.697.666-.728	.727.717-.737	.609.578-.640
	NaiveText	.699.684-.714	.775.754-.793	.709.674-.744	.657.646-.668	.616.592-.640
	Local-Rubric	.772.758-.784	.802.786-.818	.770.738-.799	.789.780-.798	.606.583-.630
	Local-Rubric-Basic	.737.723-.752	.783.765-.800	.725.692-.758	.752.742-.761	.599.568-.628
	Local-Rubric-NoInterp	.770.757-.782	.797.779-.815	.776.747-.803	.780.771-.789	.596.565-.625
	Global-Rubric-Blind	.751.738-.764	.776.755-.794	.745.716-.775	.777.768-.787	.585.559-.608
	Global-Rubric	.763.748-.777	.786.768-.805	.756.723-.789	.791.781-.800	.594.567-.619
$n = \text{All}$	Global-Rubric-Auto	.756.743-.769	.790.772-.807	.752.722-.782	.776.766-.786	.575.543-.605
	Global-Rubric-Tabular	.752.740-.764	.766.748-.785	.737.710-.763	.804.794-.813	.540.507-.572
		(b) AUPRC				
		Overall (15)	Oper. (3)	New Dx (6)	Labs (5)	CXR (1)
$n = 0$	Qwen3-8B-Zeroshot	.316.309-.324	.185.172-.200	.080.069-.093	.637.623-.650	.526.491-.560
	GPT5-Mini-Zeroshot	.348.337-.362	.221.205-.240	.137.111-.169	.647.634-.659	.507.476-.538
$n = 10$	Count-GBM	.298.289-.307	.213.193-.235	.098.084-.116	.540.526-.555	.543.502-.583
	CLMBR-T	.315.305-.326	.265.237-.294	.099.084-.116	.575.561-.590	.465.427-.500
	NaiveText	.306.296-.316	.225.200-.253	.097.081-.114	.557.541-.572	.551.519-.583
	Local-Rubric	.382.369-.396	.270.244-.296	.170.143-.198	.680.664-.694	.503.474-.535
	Local-Rubric-Basic	.345.333-.359	.219.199-.240	.163.136-.193	.610.595-.624	.493.454-.531
	Local-Rubric-NoInterp	.371.356-.387	.257.232-.284	.175.144-.206	.649.634-.663	.509.469-.546
	Global-Rubric-Blind	.333.322-.345	.207.189-.230	.133.112-.155	.625.609-.639	.455.427-.486
	Global-Rubric	.345.332-.359	.241.219-.264	.147.120-.177	.610.595-.625	.521.489-.552
	Global-Rubric-Auto	.341.331-.353	.254.229-.283	.120.104-.139	.626.610-.642	.497.460-.534
$n = \text{All}$	Global-Rubric-Tabular	.303.296-.310	.184.169-.200	.091.081-.102	.589.576-.603	.497.470-.523
	Count-GBM	.371.353-.388	.274.250-.302	.218.180-.255	.570.556-.584	.582.540-.624
	CLMBR-T	.430.417-.444	.425.387-.468	.170.145-.197	.713.699-.726	.600.559-.643
	NaiveText	.391.377-.406	.315.283-.346	.179.151-.208	.649.634-.664	.609.577-.641
	Local-Rubric	.452.439-.466	.341.310-.374	.223.194-.251	.762.748-.776	.605.571-.638
	Local-Rubric-Basic	.431.417-.445	.333.304-.368	.203.174-.234	.731.716-.744	.594.554-.633
	Local-Rubric-NoInterp	.451.437-.465	.345.315-.379	.227.198-.257	.758.744-.771	.581.537-.625
	Global-Rubric-Blind	.428.415-.443	.321.292-.351	.185.155-.217	.753.739-.767	.587.556-.620
	Global-Rubric	.459.442-.478	.339.309-.371	.236.200-.276	.773.760-.786	.582.550-.614
$n = \text{All}$	Global-Rubric-Auto	.437.422-.452	.343.312-.378	.195.164-.226	.760.745-.773	.557.518-.596
	Global-Rubric-Tabular	.445.429-.461	.329.296-.363	.208.175-.243	.783.769-.796	.527.486-.567

E.2 Results using different text-embedding models as backbones in downstream training

Table 3: Performance with different text-embedding models ($n = \text{All}$). **Green**: best in entire column. **Blue**: best within a method across backbones. **Oper.:** Operational Outcomes, **New Dx:** Assignment of New Diagnosis, **Labs:** Anticipating Lab Results, **CXR:** Chest X-ray Findings.

(A) AUROC						
Method	Backbone	Overall	Oper. (3)	New Dx (6)	Labs (5)	CXR (1)
NaiveText	Mistral-7B	.612.596-.626	.656.631-.681	.647.610-.681	.574.562-.585	.453.421-.483
	Llama3-8B	.594.578-.609	.660.633-.687	.619.584-.654	.552.540-.563	.463.433-.494
	TE3-L	.674.660-.689	.741.719-.761	.713.681-.744	.610.599-.621	.564.532-.596
	Qwen3-8B	.699.684-.714	.775.754-.793	.709.674-.744	.657.646-.668	.616.592-.640
Local-Rubric	Mistral-7B	.760.746-.773	.804.787-.820	.762.727-.794	.768.759-.778	.569.537-.598
	Llama3-8B	.761.746-.775	.793.774-.810	.764.726-.796	.772.762-.780	.591.560-.621
	TE3-L	.762.749-.774	.798.781-.815	.762.732-.790	.782.773-.790	.553.523-.585
	Qwen3-8B	.772.758-.784	.802.786-.818	.770.738-.799	.789.780-.798	.606.583-.630
Global-Rubric	Mistral-7B	.725.711-.739	.762.742-.782	.692.659-.724	.775.766-.785	.557.532-.583
	Llama3-8B	.736.722-.750	.764.744-.784	.715.684-.747	.777.768-.787	.565.540-.590
	TE3-L	.738.726-.750	.764.743-.784	.714.687-.742	.788.778-.796	.561.537-.584
	Qwen3-8B	.763.748-.777	.786.768-.805	.756.723-.789	.791.781-.800	.594.567-.619
(B) AUPRC						
Method	Backbone	Overall	Oper. (3)	New Dx (6)	Labs (5)	CXR (1)
NaiveText	Mistral-7B	.316.304-.328	.244.220-.274	.124.103-.147	.561.546-.576	.457.424-.489
	Llama3-8B	.310.301-.321	.241.218-.269	.116.099-.133	.549.534-.563	.494.455-.532
	TE3-L	.351.340-.364	.282.254-.311	.145.122-.173	.601.586-.615	.546.508-.584
	Qwen3-8B	.391.377-.406	.315.283-.346	.179.151-.208	.649.634-.664	.609.577-.641
Local-Rubric	Mistral-7B	.446.430-.463	.357.326-.389	.224.188-.259	.741.727-.754	.573.533-.613
	Llama3-8B	.450.433-.467	.342.310-.376	.235.199-.270	.745.731-.759	.584.546-.624
	TE3-L	.445.431-.461	.348.315-.380	.218.187-.252	.755.741-.768	.559.518-.599
	Qwen3-8B	.452.439-.466	.341.310-.374	.223.194-.251	.762.748-.776	.605.571-.638
Global-Rubric	Mistral-7B	.417.404-.430	.338.307-.370	.153.129-.178	.755.742-.767	.548.516-.579
	Llama3-8B	.430.417-.445	.337.307-.368	.179.153-.208	.760.748-.773	.565.533-.598
	TE3-L	.433.419-.447	.346.311-.380	.178.153-.205	.771.758-.784	.531.502-.563
	Qwen3-8B	.459.442-.478	.339.309-.371	.236.200-.276	.773.760-.786	.582.550-.614

We evaluate four text embedding models for the downstream learning step, keeping the classifier (logistic regression on frozen embeddings) fixed.

We compare **Qwen3-Embedding-8B** [69], two **LLM2Vec** [5] bidirectional adaptations of **Llama-3-8B-Instruct** [33] and **Mistral-7B-Instruct** [18], and the proprietary **text-embedding-3-large** model (**TE3-L**) accessed through the OpenAI API [35]. We use $n = \text{All}$ and compare the main textual representation-based methods: **NaiveText**, **Local-Rubric**, and **Global-Rubric**.

As shown in Table 3, Qwen3-Embedding-8B achieves the highest overall AUROC and AUPRC across methods compared. Within different task types, it continues to be either the best text-embedding model, or very close to top model. In the remainder of the paper, we report results for Qwen3-Embedding-8B model, unless otherwise stated.

E.3 Sample-size sweep figures with confidence bands

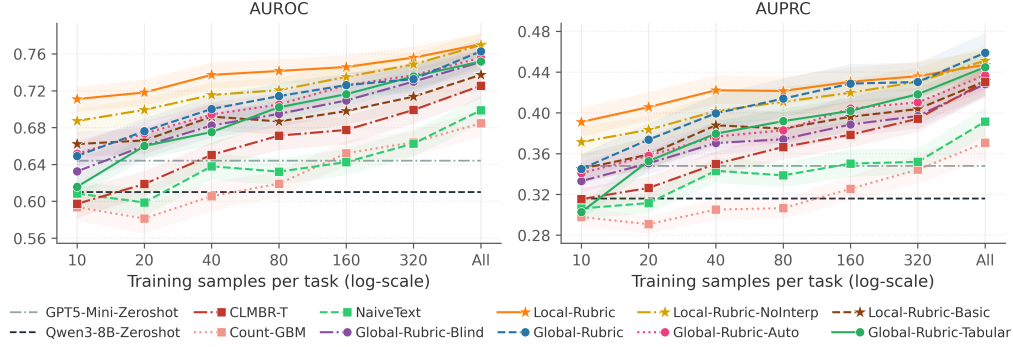


Figure 9: Average overall performance. Bands denote 95% CI, bootstrapped over the test set.

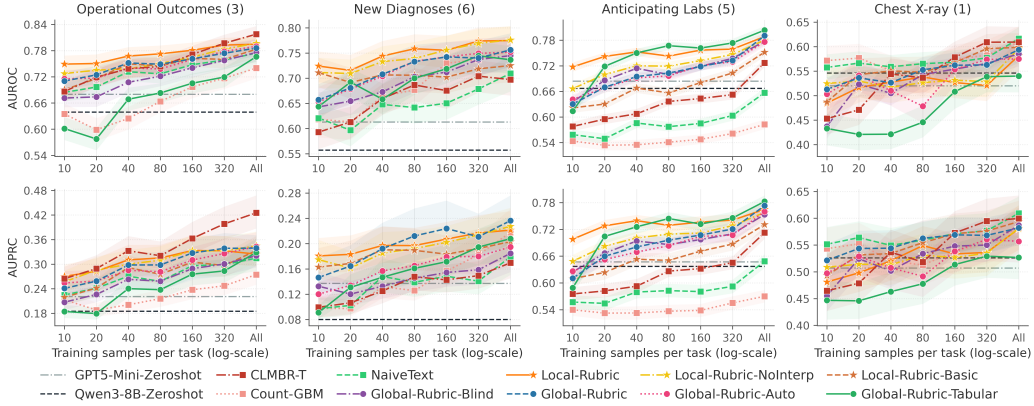


Figure 10: Average performance per task group. Bands denote 95% CI, bootstrapped over the test set.

E.4 Full EHRSHOT evaluation results for Global-Rubric-Tabular

Evaluating [Global-Rubric-Tabular](#) on the full EHRSHOT dataset without subsampling is feasible as there are no per-example LLM calls needed. Table 4 reports results for the full dataset. The model achieves a mean AUROC of 0.770 and mean AUPRC of 0.312 across tasks.

Table 4: Full-dataset EHRSHOT results for [Global-Rubric-Tabular](#).

Task	AUROC	AUPRC	Task	AUROC	AUPRC
<i>Operational Outcomes</i>			<i>Anticipating Lab Results</i>		
ICU transfer	.800 _{.752-.846}	.195 _{.130-.277}	Anemia	.810 _{.806-.815}	.361 _{.351-.372}
Long length of stay	.737 _{.714-.758}	.464 _{.421-.509}	Hyponatremia	.815 _{.811-.818}	.560 _{.552-.568}
30-day readmit	.757 _{.725-.787}	.357 _{.300-.419}	Thrombocytopenia	.885 _{.881-.888}	.571 _{.558-.582}
Group Avg.	.765_{.745-.786}	.340_{.308-.374}	Hyperkalemia	.853 _{.840-.865}	.097 _{.086-.109}
<i>Assignment of New Diagnoses</i>			Hypoglycemia	.751 _{.733-.768}	.031 _{.026-.037}
Acute MI	.765 _{.726-.803}	.183 _{.140-.225}	Group Avg.	.823_{.818-.827}	.324_{.319-.329}
Lupus	.819 _{.728-.897}	.048 _{.020-.078}	<i>Chest X-ray Findings</i>		
Hyperlipidemia	.736 _{.697-.772}	.312 _{.248-.383}	Chest X-ray	.584 _{.572-.597}	.743 _{.731-.756}
Hypertension	.718 _{.677-.757}	.299 _{.239-.368}	Group Avg.	.584_{.571-.597}	.743_{.731-.756}
Celiac disease	.654 _{.540-.772}	.098 _{.016-.190}	Overall Avg. (15 tasks)	.770_{.756-.783}	.312_{.297-.327}
Pancreatic cancer	.859 _{.798-.915}	.378 _{.239-.518}			
Group Avg.	.759_{.729-.790}	.216_{.185-.248}			

E.5 Downstream classifier hyperparameters

For all logistic-regression downstream classifiers (textual rubric variants, [NaiveText](#), and [CLMBR-T](#)), we use scikit-learn’s `LogisticRegression` with an L2 penalty. The inverse regularization strength C is tuned, for each task and each training-set size n , on the validation split using negative log-likelihood, sweeping a log-spaced grid of 13 values, $C \in \text{np.logspace}(-6, -1, 13)$ (i.e., $C \in \{10^{-6}, \dots, 10^{-1}\}$). The maximum number of iterations is set high enough for convergence on all tasks. We enable class-balanced sample reweighting.

For [Global-Rubric-Tabular](#), we train an `xgboost.XGBClassifier` and tune the number of trees, maximum depth, learning rate, and minimum child weight on the validation split using negative log-likelihood. The grid is:

- `n_estimators` $\in \{50, 100, 300\}$
- `max_depth` $\in \{2, 3, 5, 7\}$
- `learning_rate` $\in \{0.01, 0.05\}$
- `min_child_weight` $\in \{1, 5\}$
- `subsample` = 0.8 (fixed)

For [Count-GBM](#), we train a `lightgbm.LGBMClassifier`, and for [CLMBR-T](#) a logistic head over embeddings, following the search procedure of Wornow et al. [61].

E.6 Token counts of different textual representations

Table 5: Token count statistics for the textual representations that are passed into the text embedding model prior to downstream learning across different methods. Statistics for each representation are computed after merging data from all 15 tasks and 3 splits in Table 1. Qwen3-8B tokenizer is used.

Representation	Mean	Median	Min	Max	Std
NaiveText	5,980	7,769	131	8,267	2,648
Local-Rubric	853	844	492	1,430	116
Global-Rubric	1,283	1,193	318	3,136	452

E.7 Per-task results

In this section, we report AUROC and AUPRC metrics separately for each task in Figures 11–14 and Tables 6–20. Qwen3-Embedding-8B is used as the text-embedding model. For some results with other embedding models, see Appendix E.2.

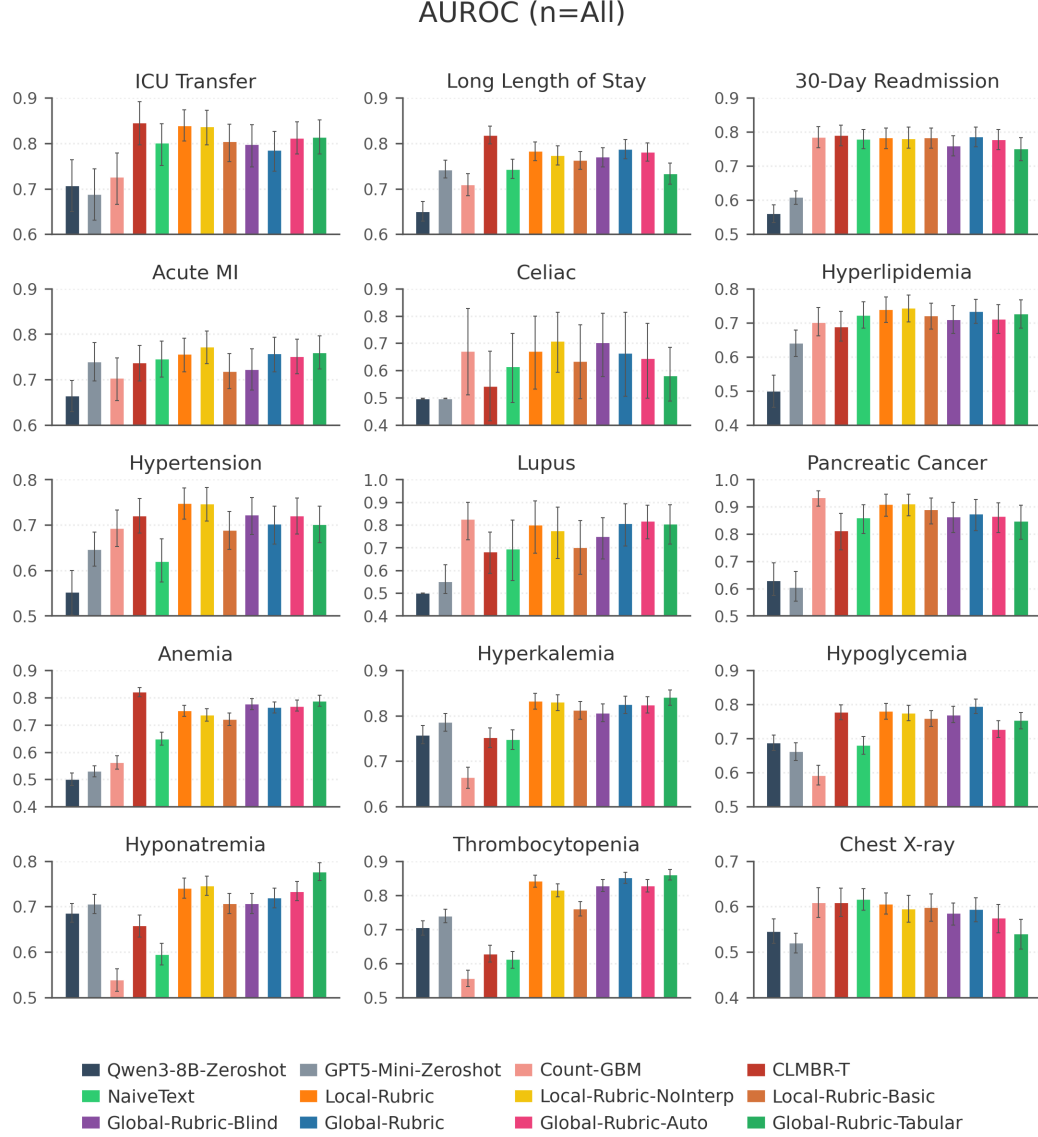


Figure 11: AUROC per-task, $n = \text{All}$

AUPRC (n=All)

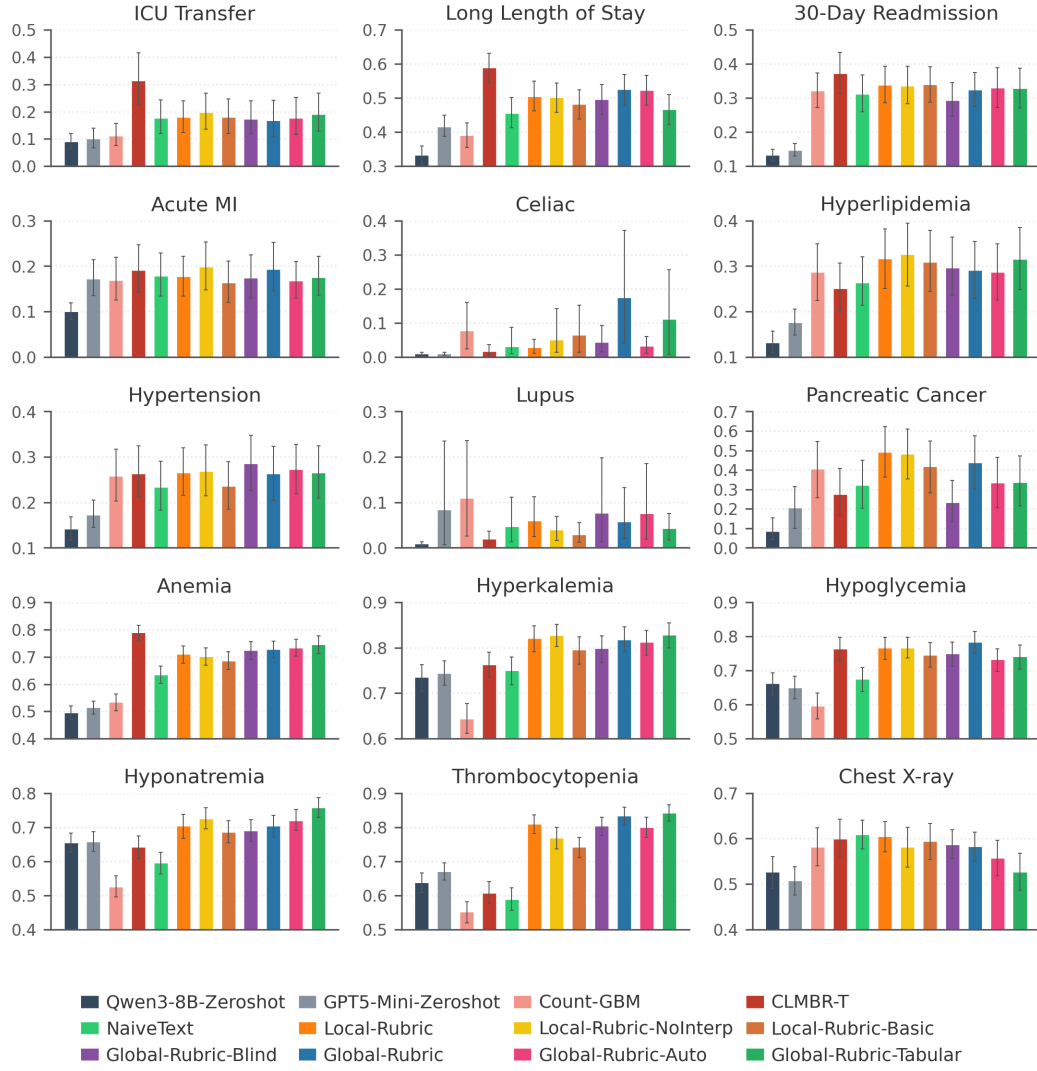


Figure 12: AUPRC per-task, $n = \text{All}$

AUROC (n=10)



Figure 13: AUROC per-task, $n = 10$

AUPRC (n=10)

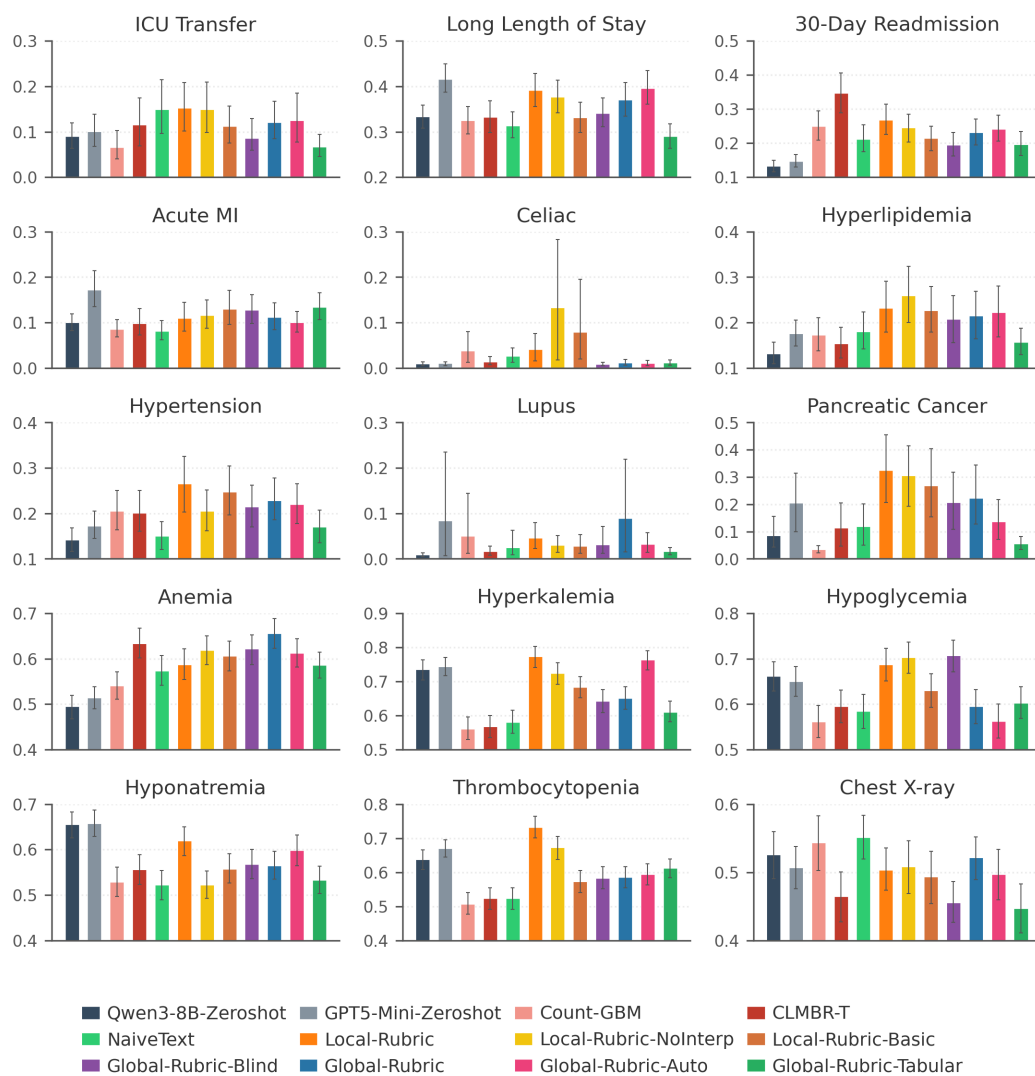


Figure 14: AUPRC per-task, $n = 10$

Table 6: Guo ICU. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.707 _{.650–.764}		.090 _{.064–.120}	
GPT5-Mini-Zeroshot	.688 _{.631–.744}		.101 _{.068–.139}	
	$n = 10$	$n = \text{All}$	$n = 10$	$n = \text{All}$
Count-GBM	.564 _{.508–.622}	.726 _{.665–.779}	.066 _{.040–.103}	.112 _{.076–.157}
CLMBR-T	.678 _{.617–.736}	.845 _{.797–.892}	.115 _{.069–.174}	.314 _{.226–.416}
NaiveText	.759 _{.711–.807}	.801 _{.751–.843}	.149 _{.097–.214}	.177 _{.119–.243}
Local-Rubric	.789 _{.745–.832}	.839 _{.805–.873}	.152 _{.102–.209}	.179 _{.124–.239}
Local-Rubric-Basic	.731 _{.681–.775}	.804 _{.759–.842}	.113 _{.075–.156}	.179 _{.120–.247}
Local-Rubric-NoInterp	.783 _{.739–.826}	.837 _{.797–.873}	.150 _{.098–.209}	.197 _{.136–.267}
Global-Rubric-Blind	.692 _{.642–.740}	.797 _{.748–.841}	.086 _{.059–.130}	.173 _{.118–.240}
Global-Rubric	.730 _{.680–.784}	.785 _{.738–.827}	.121 _{.085–.167}	.168 _{.108–.242}
Global-Rubric-Auto	.719 _{.671–.766}	.811 _{.777–.847}	.125 _{.077–.185}	.177 _{.116–.252}
Global-Rubric-Tabular	.591 _{.529–.653}	.814 _{.777–.851}	.067 _{.045–.095}	.191 _{.128–.268}

Table 7: Guo length of stay. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.650 _{.629–.672}		.333 _{.308–.359}	
GPT5-Mini-Zeroshot	.742 _{.724–.763}		.416 _{.387–.449}	
	$n = 10$	$n = \text{All}$	$n = 10$	$n = \text{All}$
Count-GBM	.628 _{.604–.654}	.709 _{.685–.733}	.325 _{.295–.355}	.390 _{.355–.426}
CLMBR-T	.611 _{.587–.637}	.818 _{.799–.837}	.332 _{.298–.368}	.589 _{.546–.632}
NaiveText	.616 _{.588–.641}	.743 _{.723–.765}	.313 _{.286–.344}	.455 _{.413–.502}
Local-Rubric	.721 _{.698–.744}	.783 _{.762–.803}	.391 _{.356–.428}	.505 _{.462–.550}
Local-Rubric-Basic	.633 _{.606–.659}	.763 _{.743–.782}	.331 _{.299–.365}	.482 _{.438–.524}
Local-Rubric-NoInterp	.683 _{.659–.707}	.773 _{.752–.794}	.376 _{.342–.414}	.502 _{.458–.544}
Global-Rubric-Blind	.661 _{.637–.686}	.770 _{.748–.791}	.341 _{.311–.374}	.496 _{.450–.540}
Global-Rubric	.675 _{.649–.698}	.787 _{.766–.809}	.370 _{.334–.408}	.526 _{.480–.570}
Global-Rubric-Auto	.689 _{.664–.714}	.781 _{.761–.801}	.396 _{.361–.434}	.523 _{.479–.567}
Global-Rubric-Tabular	.557 _{.533–.582}	.734 _{.710–.757}	.290 _{.264–.317}	.467 _{.423–.510}

Table 8: Guo readmission. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.561 _{.534–.586}		.133 _{.116–.150}	
GPT5-Mini-Zeroshot	.609 _{.588–.627}		.147 _{.130–.166}	
	$n = 10$	$n = \text{All}$	$n = 10$	$n = \text{All}$
Count-GBM	.712 _{.676–.748}	.785 _{.754–.816}	.249 _{.208–.295}	.321 _{.272–.373}
CLMBR-T	.771 _{.738–.800}	.791 _{.760–.819}	.347 _{.289–.405}	.373 _{.313–.434}
NaiveText	.678 _{.645–.711}	.780 _{.751–.808}	.212 _{.176–.253}	.312 _{.259–.368}
Local-Rubric	.739 _{.707–.771}	.783 _{.752–.812}	.267 _{.226–.315}	.339 _{.286–.393}
Local-Rubric-Basic	.707 _{.677–.739}	.783 _{.752–.811}	.214 _{.178–.250}	.339 _{.288–.392}
Local-Rubric-NoInterp	.717 _{.684–.747}	.781 _{.752–.814}	.245 _{.203–.285}	.335 _{.283–.393}
Global-Rubric-Blind	.662 _{.628–.693}	.760 _{.729–.788}	.195 _{.162–.231}	.293 _{.246–.346}
Global-Rubric	.726 _{.695–.753}	.780 _{.757–.814}	.232 _{.195–.271}	.324 _{.275–.375}
Global-Rubric-Auto	.739 _{.708–.766}	.778 _{.748–.807}	.242 _{.206–.282}	.329 _{.272–.390}
Global-Rubric-Tabular	.656 _{.619–.692}	.751 _{.715–.783}	.196 _{.163–.233}	.328 _{.271–.387}

Table 9: Acute MI. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.664 _{.629–.698}		.100 _{.082–.119}	
GPT5-Mini-Zeroshot	.739 _{.697–.782}		.172 _{.135–.214}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.572 _{.520–.622}	.704 _{.654–.748}	.086 _{.068–.106}	.169 _{.125–.219}
CLMBR-T	.577 _{.528–.621}	.737 _{.697–.775}	.099 _{.073–.131}	.191 _{.142–.247}
NaiveText	.520 _{.470–.568}	.746 _{.706–.785}	.081 _{.062–.105}	.179 _{.134–.229}
Local-Rubric	.618 _{.570–.665}	.756 _{.717–.791}	.110 _{.081–.144}	.177 _{.134–.222}
Local-Rubric-Basic	.633 _{.583–.683}	.718 _{.680–.757}	.129 _{.096–.171}	.163 _{.120–.211}
Local-Rubric-NoInterp	.616 _{.564–.667}	.772 _{.735–.807}	.116 _{.088–.150}	.199 _{.148–.253}
Global-Rubric-Blind	.687 _{.643–.732}	.723 _{.677–.768}	.128 _{.098–.161}	.174 _{.130–.225}
Global-Rubric	.623 _{.572–.669}	.757 _{.717–.793}	.112 _{.084–.144}	.194 _{.146–.253}
Global-Rubric-Auto	.631 _{.585–.675}	.751 _{.713–.789}	.100 _{.079–.125}	.168 _{.130–.211}
Global-Rubric-Tabular	.695 _{.649–.737}	.760 _{.723–.797}	.134 _{.107–.165}	.176 _{.136–.221}

Table 10: Celiac disease. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.497 _{.495–.498}		.009 _{.005–.014}	
GPT5-Mini-Zeroshot	.497 _{.496–.499}		.010 _{.005–.014}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.603 _{.457–.745}	.671 _{.510–.828}	.038 _{.012–.080}	.077 _{.024–.160}
CLMBR-T	.532 _{.409–.642}	.543 _{.415–.671}	.013 _{.007–.025}	.017 _{.007–.036}
NaiveText	.712 _{.599–.810}	.614 _{.482–.736}	.026 _{.012–.044}	.030 _{.009–.087}
Local-Rubric	.680 _{.515–.828}	.670 _{.532–.799}	.041 _{.016–.076}	.028 _{.012–.052}
Local-Rubric-Basic	.757 _{.671–.849}	.634 _{.497–.768}	.079 _{.020–.195}	.064 _{.014–.152}
Local-Rubric-NoInterp	.704 _{.567–.826}	.708 _{.594–.813}	.132 _{.018–.283}	.050 _{.014–.141}
Global-Rubric-Blind	.406 _{.304–.514}	.702 _{.578–.810}	.009 _{.005–.013}	.043 _{.015–.092}
Global-Rubric	.546 _{.428–.646}	.663 _{.505–.814}	.012 _{.007–.019}	.174 _{.040–.372}
Global-Rubric-Auto	.480 _{.359–.609}	.644 _{.499–.774}	.011 _{.006–.017}	.032 _{.012–.060}
Global-Rubric-Tabular	.570 _{.474–.660}	.582 _{.487–.686}	.012 _{.007–.017}	.111 _{.008–.255}

Table 11: Hyperlipidemia. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.500 _{.452–.546}		.132 _{.109–.157}	
GPT5-Mini-Zeroshot	.640 _{.601–.678}		.176 _{.148–.205}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.546 _{.496–.597}	.702 _{.662–.745}	.172 _{.138–.211}	.287 _{.225–.349}
CLMBR-T	.512 _{.466–.560}	.689 _{.647–.733}	.153 _{.122–.189}	.251 _{.202–.307}
NaiveText	.559 _{.510–.613}	.722 _{.684–.762}	.180 _{.142–.223}	.263 _{.213–.320}
Local-Rubric	.581 _{.531–.632}	.740 _{.701–.776}	.232 _{.179–.291}	.316 _{.251–.382}
Local-Rubric-Basic	.619 _{.571–.666}	.721 _{.681–.758}	.226 _{.179–.280}	.309 _{.245–.379}
Local-Rubric-NoInterp	.617 _{.568–.667}	.743 _{.703–.782}	.259 _{.200–.323}	.326 _{.256–.394}
Global-Rubric-Blind	.542 _{.491–.593}	.710 _{.668–.750}	.208 _{.156–.259}	.297 _{.236–.364}
Global-Rubric	.567 _{.516–.620}	.734 _{.698–.769}	.215 _{.165–.269}	.291 _{.228–.355}
Global-Rubric-Auto	.589 _{.540–.639}	.711 _{.669–.753}	.222 _{.168–.280}	.286 _{.225–.349}
Global-Rubric-Tabular	.572 _{.529–.612}	.727 _{.684–.768}	.157 _{.129–.187}	.316 _{.248–.385}

Table 12: Hypertension. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.552 _{.502–.600}		.142 _{.116–.168}	
GPT5-Mini-Zeroshot	.646 _{.609–.684}		.172 _{.145–.205}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.662 _{.622–.704}	.693 _{.653–.732}	.206 _{.164–.251}	.258 _{.203–.317}
CLMBR-T	.653 _{.611–.696}	.721 _{.682–.759}	.201 _{.160–.251}	.263 _{.213–.324}
NaiveText	.566 _{.525–.609}	.620 _{.574–.669}	.150 _{.121–.182}	.234 _{.183–.291}
Local-Rubric	.709 _{.669–.747}	.747 _{.713–.782}	.265 _{.203–.325}	.265 _{.215–.320}
Local-Rubric-Basic	.707 _{.664–.747}	.688 _{.646–.730}	.247 _{.196–.305}	.236 _{.185–.290}
Local-Rubric-NoInterp	.665 _{.625–.702}	.746 _{.709–.782}	.205 _{.162–.251}	.268 _{.215–.326}
Global-Rubric-Blind	.681 _{.637–.721}	.722 _{.679–.761}	.214 _{.170–.262}	.285 _{.226–.347}
Global-Rubric	.693 _{.653–.736}	.702 _{.658–.742}	.228 _{.186–.278}	.263 _{.204–.323}
Global-Rubric-Auto	.696 _{.658–.735}	.721 _{.680–.759}	.220 _{.178–.265}	.272 _{.219–.327}
Global-Rubric-Tabular	.600 _{.547–.649}	.701 _{.661–.742}	.171 _{.135–.208}	.266 _{.209–.324}

Table 13: Lupus. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.500 _{.499–.500}		.009 _{.005–.013}	
GPT5-Mini-Zeroshot	.550 _{.499–.625}		.084 _{.007–.235}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.754 _{.637–.849}	.826 _{.734–.901}	.051 _{.012–.144}	.109 _{.026–.236}
CLMBR-T	.614 _{.486–.747}	.681 _{.588–.768}	.016 _{.008–.028}	.020 _{.009–.037}
NaiveText	.670 _{.575–.767}	.695 _{.556–.822}	.025 _{.009–.063}	.047 _{.013–.112}
Local-Rubric	.849 _{.780–.907}	.801 _{.676–.906}	.046 _{.022–.079}	.060 _{.024–.113}
Local-Rubric-Basic	.720 _{.602–.831}	.701 _{.584–.819}	.028 _{.012–.054}	.029 _{.012–.056}
Local-Rubric-NoInterp	.788 _{.713–.865}	.775 _{.652–.879}	.030 _{.015–.051}	.040 _{.017–.069}
Global-Rubric-Blind	.731 _{.627–.821}	.750 _{.650–.831}	.031 _{.012–.071}	.076 _{.013–.198}
Global-Rubric	.747 _{.648–.840}	.807 _{.708–.892}	.089 _{.016–.220}	.058 _{.020–.132}
Global-Rubric-Auto	.804 _{.727–.872}	.818 _{.739–.886}	.033 _{.015–.058}	.075 _{.020–.186}
Global-Rubric-Tabular	.727 _{.668–.760}	.804 _{.716–.889}	.017 _{.009–.025}	.043 _{.017–.076}

Table 14: Pancreatic cancer. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.630 _{.574–.694}		.086 _{.043–.155}	
GPT5-Mini-Zeroshot	.604 _{.554–.662}		.205 _{.099–.314}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.574 _{.503–.644}	.933 _{.902–.959}	.034 _{.023–.049}	.406 _{.258–.546}
CLMBR-T	.669 _{.589–.740}	.812 _{.741–.876}	.114 _{.046–.205}	.276 _{.165–.407}
NaiveText	.697 _{.630–.766}	.860 _{.803–.908}	.120 _{.051–.201}	.321 _{.202–.449}
Local-Rubric	.843 _{.776–.900}	.909 _{.865–.947}	.325 _{.206–.454}	.491 _{.362–.622}
Local-Rubric-Basic	.829 _{.757–.886}	.889 _{.836–.932}	.268 _{.154–.404}	.417 _{.283–.548}
Local-Rubric-NoInterp	.878 _{.826–.924}	.911 _{.866–.946}	.306 _{.192–.414}	.482 _{.353–.611}
Global-Rubric-Blind	.812 _{.755–.864}	.864 _{.805–.916}	.207 _{.109–.318}	.232 _{.134–.346}
Global-Rubric	.767 _{.691–.839}	.874 _{.813–.928}	.223 _{.127–.344}	.438 _{.301–.575}
Global-Rubric-Auto	.718 _{.641–.794}	.866 _{.806–.915}	.136 _{.071–.217}	.335 _{.205–.464}
Global-Rubric-Tabular	.699 _{.633–.763}	.847 _{.780–.906}	.056 _{.035–.081}	.337 _{.217–.473}

Table 15: Anemia. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.500 _{.478–.523}		.495 _{.468–.520}	
GPT5-Mini-Zeroshot	.530 _{.510–.549}		.514 _{.490–.538}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.563 _{.538–.587}	.562 _{.537–.587}	.540 _{.511–.571}	.533 _{.502–.563}
CLMBR-T	.646 _{.624–.669}	.821 _{.803–.837}	.634 _{.602–.668}	.788 _{.759–.816}
NaiveText	.581 _{.554–.605}	.649 _{.625–.673}	.573 _{.541–.607}	.634 _{.602–.667}
Local-Rubric	.584 _{.559–.609}	.752 _{.731–.773}	.588 _{.554–.622}	.710 _{.677–.740}
Local-Rubric-Basic	.618 _{.593–.641}	.721 _{.698–.743}	.606 _{.573–.639}	.686 _{.653–.719}
Local-Rubric-NoInterp	.620 _{.596–.644}	.736 _{.714–.758}	.619 _{.587–.650}	.701 _{.669–.733}
Global-Rubric-Blind	.662 _{.638–.686}	.776 _{.756–.797}	.622 _{.587–.652}	.724 _{.691–.757}
Global-Rubric	.680 _{.657–.702}	.764 _{.744–.785}	.656 _{.623–.689}	.728 _{.696–.758}
Global-Rubric-Auto	.638 _{.615–.661}	.769 _{.750–.790}	.613 _{.582–.644}	.734 _{.703–.764}
Global-Rubric-Tabular	.621 _{.598–.644}	.787 _{.768–.809}	.586 _{.558–.614}	.745 _{.712–.776}

Table 16: Hyperkalemia. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.758 _{.738–.779}		.735 _{.705–.763}	
GPT5-Mini-Zeroshot	.786 _{.766–.805}		.744 _{.717–.771}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.568 _{.543–.593}	.665 _{.640–.687}	.561 _{.530–.595}	.644 _{.612–.677}
CLMBR-T	.562 _{.536–.588}	.752 _{.730–.774}	.568 _{.535–.601}	.763 _{.735–.791}
NaiveText	.592 _{.566–.617}	.748 _{.726–.769}	.581 _{.548–.616}	.750 _{.719–.779}
Local-Rubric	.782 _{.761–.803}	.832 _{.814–.850}	.773 _{.740–.803}	.821 _{.791–.848}
Local-Rubric-Basic	.679 _{.654–.702}	.812 _{.792–.831}	.683 _{.652–.715}	.795 _{.764–.824}
Local-Rubric-NoInterp	.735 _{.712–.758}	.830 _{.812–.846}	.724 _{.691–.755}	.827 _{.803–.852}
Global-Rubric-Blind	.660 _{.635–.685}	.806 _{.787–.826}	.643 _{.608–.676}	.798 _{.767–.826}
Global-Rubric	.681 _{.656–.706}	.825 _{.805–.844}	.651 _{.618–.684}	.818 _{.791–.846}
Global-Rubric-Auto	.760 _{.738–.780}	.824 _{.806–.842}	.764 _{.735–.790}	.812 _{.784–.838}
Global-Rubric-Tabular	.635 _{.610–.659}	.840 _{.823–.857}	.611 _{.581–.642}	.828 _{.800–.855}

Table 17: Hypoglycemia. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.687 _{.665–.710}		.662 _{.629–.693}	
GPT5-Mini-Zeroshot	.662 _{.636–.687}		.650 _{.617–.683}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.571 _{.543–.598}	.591 _{.563–.621}	.561 _{.526–.597}	.596 _{.557–.634}
CLMBR-T	.611 _{.584–.639}	.777 _{.755–.799}	.595 _{.560–.631}	.764 _{.730–.797}
NaiveText	.587 _{.559–.616}	.680 _{.654–.706}	.584 _{.546–.621}	.675 _{.638–.709}
Local-Rubric	.670 _{.644–.698}	.780 _{.757–.803}	.687 _{.651–.723}	.767 _{.732–.798}
Local-Rubric-Basic	.642 _{.615–.668}	.759 _{.736–.782}	.630 _{.593–.667}	.745 _{.710–.781}
Local-Rubric-NoInterp	.709 _{.683–.734}	.775 _{.753–.797}	.703 _{.668–.737}	.767 _{.736–.798}
Global-Rubric-Blind	.685 _{.656–.713}	.769 _{.746–.795}	.707 _{.671–.741}	.750 _{.713–.784}
Global-Rubric	.592 _{.565–.621}	.794 _{.774–.816}	.590 _{.557–.632}	.783 _{.750–.814}
Global-Rubric-Auto	.586 _{.558–.614}	.727 _{.703–.752}	.563 _{.525–.601}	.733 _{.697–.763}
Global-Rubric-Tabular	.641 _{.613–.669}	.754 _{.729–.776}	.603 _{.568–.638}	.741 _{.705–.775}

Table 18: Hyponatremia. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.686 _{.666–.707}		.655 _{.626–.683}	
GPT5-Mini-Zeroshot	.706 _{.685–.727}		.657 _{.629–.687}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.518 _{.491–.545}	.539 _{.514–.563}	.529 _{.497–.561}	.526 _{.495–.558}
CLMBR-T	.548 _{.522–.572}	.658 _{.633–.681}	.557 _{.523–.589}	.641 _{.608–.674}
NaiveText	.514 _{.489–.538}	.595 _{.571–.619}	.522 _{.490–.554}	.595 _{.563–.626}
Local-Rubric	.663 _{.638–.688}	.740 _{.718–.763}	.619 _{.587–.650}	.703 _{.668–.737}
Local-Rubric-Basic	.561 _{.538–.587}	.707 _{.684–.729}	.557 _{.526–.591}	.686 _{.655–.719}
Local-Rubric-NoInterp	.549 _{.525–.575}	.745 _{.725–.767}	.523 _{.493–.553}	.725 _{.695–.758}
Global-Rubric-Blind	.559 _{.534–.586}	.706 _{.684–.729}	.568 _{.536–.600}	.690 _{.659–.722}
Global-Rubric	.574 _{.550–.599}	.719 _{.697–.740}	.564 _{.535–.596}	.703 _{.672–.735}
Global-Rubric-Auto	.608 _{.583–.633}	.733 _{.713–.756}	.599 _{.565–.632}	.720 _{.691–.751}
Global-Rubric-Tabular	.517 _{.492–.541}	.776 _{.757–.797}	.533 _{.503–.564}	.757 _{.730–.787}

Table 19: Thrombocytopenia. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.705 _{.683–.725}		.637 _{.608–.666}	
GPT5-Mini-Zeroshot	.739 _{.719–.759}		.670 _{.645–.696}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.498 _{.473–.525}	.556 _{.531–.580}	.507 _{.477–.540}	.551 _{.519–.582}
CLMBR-T	.522 _{.496–.548}	.627 _{.604–.654}	.523 _{.492–.554}	.606 _{.575–.640}
NaiveText	.518 _{.491–.545}	.612 _{.586–.635}	.524 _{.491–.555}	.588 _{.556–.622}
Local-Rubric	.771 _{.749–.791}	.841 _{.824–.859}	.732 _{.701–.765}	.810 _{.781–.836}
Local-Rubric-Basic	.608 _{.584–.632}	.760 _{.739–.781}	.572 _{.541–.606}	.742 _{.711–.770}
Local-Rubric-NoInterp	.718 _{.696–.739}	.815 _{.796–.834}	.673 _{.638–.705}	.769 _{.736–.800}
Global-Rubric-Blind	.611 _{.588–.634}	.828 _{.810–.847}	.583 _{.552–.617}	.804 _{.776–.830}
Global-Rubric	.625 _{.601–.648}	.851 _{.834–.868}	.585 _{.554–.617}	.834 _{.807–.859}
Global-Rubric-Auto	.616 _{.592–.640}	.828 _{.809–.846}	.593 _{.563–.625}	.800 _{.770–.829}
Global-Rubric-Tabular	.656 _{.632–.679}	.860 _{.844–.875}	.613 _{.584–.640}	.841 _{.818–.866}

Table 20: Chest X-ray. AUROC and AUPRC with 95% bootstrap CI. Best per column for each setting is highlighted. Embeddings are from Qwen3-8B; sample sizes refer to per-class training examples.

	AUROC		AUPRC	
Qwen3-8B-Zeroshot	.546 _{.519–.573}		.526 _{.491–.560}	
GPT5-Mini-Zeroshot	.520 _{.498–.541}		.507 _{.476–.538}	
	<i>n</i> = 10	<i>n</i> = All	<i>n</i> = 10	<i>n</i> = All
Count-GBM	.571 _{.541–.602}	.609 _{.576–.642}	.543 _{.502–.583}	.582 _{.540–.624}
CLMBR-T	.453 _{.421–.484}	.609 _{.578–.640}	.465 _{.427–.500}	.600 _{.559–.643}
NaiveText	.558 _{.533–.584}	.616 _{.592–.640}	.551 _{.519–.583}	.609 _{.577–.641}
Local-Rubric	.517 _{.492–.543}	.606 _{.583–.630}	.503 _{.474–.535}	.605 _{.571–.638}
Local-Rubric-Basic	.487 _{.457–.519}	.599 _{.568–.628}	.493 _{.454–.531}	.594 _{.554–.633}
Local-Rubric-NoInterp	.525 _{.491–.556}	.590 _{.565–.625}	.509 _{.469–.546}	.581 _{.537–.625}
Global-Rubric-Blind	.437 _{.413–.461}	.585 _{.559–.608}	.455 _{.427–.486}	.587 _{.556–.620}
Global-Rubric	.513 _{.489–.538}	.594 _{.567–.619}	.521 _{.489–.552}	.582 _{.550–.614}
Global-Rubric-Auto	.503 _{.470–.534}	.575 _{.543–.605}	.497 _{.460–.534}	.557 _{.518–.596}
Global-Rubric-Tabular	.500 _{.500–.500}	.540 _{.507–.572}	.497 _{.470–.523}	.527 _{.486–.567}

F Full prompts used in rubric representation learning methods

F.1 Prompt template for computing text-embeddings of inputs

```
# Prompt template for obtaining text input embeddings for downstream training
Based on the patient's EHR below, predict: {task_query}

-- Patient EHR --
{xtext or xrubric}
-- End of EHR --

Based on the above EHR, predict: {task_query}
Respond with exactly one word: Yes or No.
```

Figure 15: Prompt for converting textual inputs to embeddings. An example task query: “Will the patient develop lupus within next year?”

For every textual representation used as input to the downstream logistic regression classifier (NaiveText and all textual rubric variants), the textual input (x^{text} or x^{rubric}) is first wrapped in the task-conditioned template shown in Figure 15 before being passed to the frozen text-embedding model. The template prepends and appends a brief task query so that the resulting embedding is conditioned on the prediction target rather than reflecting only generic input content.

E.2 Prompt for global rubric creation (Figure 3, Panel B)

```
# Prompt used with GPT-5-mini for global rubric synthesis

You are a medical expert designing a structured rubric for a clinical prediction task.

## Task
- Name: {task_name}
- Query: {task_query}

## Context
You will be given {40} labeled patient EHR examples ({20} positive, {20} negative). Another model will later use your rubric to transform new patient EHRs into structured summaries, which will then serve as input to a supervised classifier.

## What You Must Do
Study the examples below. Combine what you observe in them with your medical knowledge to design a rubric template - a set of named fields that, when filled in for any patient, produce a structured summary optimized for this prediction task.

The rubric should:
1. Be data-driven and discriminative. Identify which features, patterns, and interactions actually separate the positive and negative cases. The rubric should capture not just obvious indicators but also subtler or compound features you notice. At the same time, do not overfit to these 40 cases - use your clinical knowledge to include factors that are generally relevant even if not prominent in this sample.
2. Be structured and consistent. Every rubricified output must follow the same field names and order. For each field, specify what to extract from the EHR and how to format it. Specify what to write when data is absent.
3. Extract facts only. The evaluator filling in the rubric must extract and organize information from the EHR. It must NOT make predictions, assign risk levels, or draw conclusions.
4. Be concise. The rubric should focus on extracting information that is relevant to the task. It should not ask the evaluator to reproduce the entire EHR.

## Positive Examples (Ground Truth: Yes)
{NaiveText EHR serializations of 20 positive examples concatenated ( $x_{\text{text}}$  format)}

## Negative Examples (Ground Truth: No)
{NaiveText EHR serializations of 20 negative examples concatenated ( $x_{\text{text}}$  format)}

## Output
Output ONLY the rubric template itself - the instructions another model will follow to transform a patient EHR. No preamble, no explanation of your reasoning. The template must be self-contained and directly usable.
```

Figure 16: Prompt used with GPT-5-mini to guide global rubric creation from NaiveText serializations (x_{text}) of EHR examples.

E.3 Prompt for global rubric application (Figure 3, Panel D)

```
# Prompt used with GPT-5-mini for global rubric application

You are a medical data extraction specialist.

## Task
{task_query}

## Rubric Template (follow this exactly)
{rubric_instructions}

## Patient EHR
{ehr_text ( $x_{\text{text}}$  format)}

## Instructions
Fill in every field of the rubric template above using ONLY information from
this patient's EHR.

Rules:
- Follow the exact field order and section structure of the rubric.
- Be concise: use short phrases, numbers, and dates. Do not write
  paragraphs.
- If data for a field is not present in the EHR, write "No data".
- Do NOT add commentary, predictions, risk assessments, or conclusions.
- Do NOT include any information not found in the EHR above.

Rubric output:
```

Figure 17: Prompt used with GPT-5-mini for transforming a naive text serialized input (x^{text}) into its rubric text serialization version (x^{rubric}).

F.4 Prompt for creating a global rubric application parser (Figure 3, Panel E)

```
# Prompt used with GPT-5.2 for generating a parser script for rubric application

You are an expert Python developer and medical informaticist.

## Your Task
Write a complete, self-contained Python script that reads patient EHR serializations and fills in
a structured clinical rubric template using **deterministic string/regex parsing only** -- no LLM
API calls, no network requests.

## Clinical Task Context
- Task name: {task_name}
- Prediction query: {task_query}

## Rubric Template to Fill
The script must fill in every field defined in the following rubric instructions:
{rubric_instructions,  $\mathcal{R}$ }

## EHR Serialization Format
Below are 40 example patient EHR serializations from the training cohort, labeled by ground-truth
outcome. For each patient you are shown BOTH:
1. The raw naive text EHR serialization.
2. The LLM-produced rubric fill for that exact patient -- showing you how the fields should be
extracted from the raw text.

Use these paired examples to understand the extraction mapping precisely.
{40 paired examples of naive text serializations ( $x^{\text{text}}$ ) and LLM-filled rubric text serializations
( $x^{\text{rubric}}$ ).}

## Required Script Interface
The generated script must:
1. Accept the following command-line arguments via argparse:
- '-input_dir' : root directory of naivetext serializations - '-output_dir' : root directory for
llmrubric-parser outputs - '-task' : task name - '-splits' : one or more of 'train val test'

2. For each split, read '{input_dir}/{task}/{split}.json' -- a JSON array where each
element has:
- 'patient_id' (int) - 'prediction_time' (ISO datetime string) - 'task' (str) - 'split' (str) -
'label' (bool) - 'serialization' (str) ← the EHR text to parse

3. For each patient call 'fill_rubric(serialization: str) -> str', which:
- Extracts all rubric fields from the EHR text using regex and string operations - Returns a
filled-in rubric string that follows the exact field names, order, and format from the rubric
template above - Writes "NA" for any field whose data is absent from the EHR

4. Write output to '{output_dir}/{task}/{split}.json' -- a JSON array where each element
has:
- 'patient_id' (int) - 'prediction_time' (str) - 'task' (str) - 'split' (str) - 'label' (bool) -
'rubricified_text' (str) ← output of fill_rubric()

5. Create output directories as needed (parents=True, exist_ok=True).

6. Print progress to stdout: total patients processed per split.

## Constraints
- Use only Python standard library plus 're', 'json', 'argparse', 'pathlib', 'sys'. No third-party
packages. - No LLM API calls, network requests, external tools. - The 'fill_rubric' function
must be deterministic and handle missing data gracefully (write "NA" rather than raising
exceptions). - The script must be syntactically valid Python 3.8+. - Do NOT hardcode file
paths -- use the argparse arguments.

## Output
Output ONLY the Python script, with no explanation, no preamble, and no markdown fences. The
output must start with '#!/usr/bin/env python3' and be directly writable to a .py file.
```

Figure 18: Prompt used with GPT-5.2 to create a parser script for transforming a naive text serialized input (x^{text}) into its rubric text serialization version (x^{rubric}).

E.5 Prompt for creating a global rubric tabularization parser (Figure 3, Panel F)

```
# Prompt used with GPT-5.2 for generating a parser script to transform rubric serializations to
tabular features

You are an expert Python developer and medical informaticist.

## Your Task
Write a complete, self-contained Python featurizer script that reads rubric-formatted patient
EHR texts and converts each one into a **fixed-dimension numeric feature vector** using
deterministic string/regex parsing - no LLM calls, no network requests.

## Clinical Task Context
- Task name: {task_name}
- Prediction query: {task_query}

## Rubric Parser Source (shows all rubric field names and their text formats)
The following is the parser that generates the rubric text. Study it to understand which
fields exist and how their values are formatted in the text. This is the **ground truth** for
what fields can appear in a rubric text and how their values are formatted.

“python
{task-specific rubric parser generated via prompt in Section F.4}
”

## Reference Rubric Texts ({20} positive, {20} negative)

**Important context:** These {40} patients are the cohort that was used to *design* the rubric
itself. They are provided as examples so you can calibrate your regex patterns against actual
data.

**However**, the featurizer you write will be applied to a **much larger dataset** (thousands
of patients). Your feature extraction logic must therefore be:

- **General**: handle any value the rubric parser could plausibly produce, not just the values
seen in these 40 patients

- **Robust**: gracefully handle missing, NA, or unexpected values for every field

- **Comprehensive**: derive features from every field in the rubric, even if that field
happens to be NA for all {40} examples shown here

Use the parser source above as the authoritative specification of fields and value formats; use
the examples below to validate and calibrate your regex patterns.

{40 example text serialization in xrubric format}

## Required Script Interface

The generated script must:
1. Accept CLI arguments via argparse: - '-input_dir'
- '-output_dir'
- '-task'
- '-splits'

2. For each split, read '{input_dir}/{split}/{task}.json' - a JSON array where each
element has:
- 'patient_id' (int)
- 'label_time' (ISO datetime string)
- 'label_value' (bool)
- 'conversations' (list) - rubric text is in 'conversations[1]["content"]' between '-- Patient
EHR --' and '-- End of EHR --'

3. Implement 'def extract_features(rubric_text: str) -> dict[str, float]':
- Parse every rubric field from the text
- Return a flat dict mapping feature name -> float value
- For **numeric fields**: extract the number; if missing/NA write '0.0' and set
'{{field}}_missing = 1.0'
- For **categorical / Yes/No fields**: one-hot encode all known values; unknown/NA -> all zeros
plus a '{{field}}_missing = 1.0' indicator
- All returned values must be float (0.0 or 1.0 for binary, numeric otherwise)
- The dict must have the **same keys in the same order** for every call (fixed schema)

4. Define 'SCHEMA: list[dict]' at module level - one entry per feature with keys:
- "name": feature name (matches key in extract_features output)
- "type": "numeric", "binary", or "categorical"
```



```

- "description": short human-readable description
- "possible_values": list of string values for categorical/binary fields, omit for numeric

5. For each split, build an N×F float32 matrix from 'extract_features', save as:
- '{{output_dir}}/{{task}}/{{split}}.npz' with numpy keys:
- 'embeddings': shape (N, F) float32
- 'labels': shape (N,) int32
- 'patient_ids': shape (N,) int64
- 'prediction_times': shape (N,) object (strings)

6. Save '{{output_dir}}/{{task}}/feature_schema.json' once (after processing the first split):
```json
{
 "task": "{{task}}",
 "task_query": "{{task_query}}",
 "num_features": <F>,
 "features": <SCHEMA list>
}
```

7. Create output directories as needed. Print progress to stdout.

## Constraints
- Use only Python standard library plus 're', 'json', 'numpy', 'argparse', 'pathlib', 'sys'.
- No third-party packages beyond numpy.
- No LLM API calls, no network requests.
- 'extract_features' must be deterministic and never raise exceptions on any input (catch all errors, default to 0.0).
- The script must be syntactically valid Python 3.8+.
- Do NOT hardcode file paths - use the argparse arguments.
- Aim for **at least 30 features** to capture the richness of the rubric.
- Include all numeric fields, all categorical fields (one-hot), and Yes/No procedure/comorbidity flags.

## Output
Output ONLY the Python script, with no explanation, no preamble, and no markdown fences. Start with '#!/usr/bin/env python3'.
```

Figure 19: Prompt used with GPT-5.2 to create a parser script that transforms rubric-transformed inputs (x^{rubric}) into a fixed-dimensional tabular feature vector.

E.6 Prompt for stripping away interpretation of evidence from a local rubric representation (Figure 4, Left)

```
# Prompt used with GPT-5-mini for stripping away interpretive language from
Local-Rubrics
```

```
Below is a clinical summary of a patient's EHR. Your task is to produce
a MINIMALLY EDITED version that removes ONLY interpretive and predictive
language.

-- START OF EHR DATA --
{NaiveText_Serialization ( $x^{\text{text}}$ )}
-- END OF EHR DATA --

REMOVE (by deleting or replacing with neutral phrasing):

- Causal/risk language ("increases risk", "raises concern", "associated with
worse outcomes")
- Protective framing ("protective", "favorable", "reassuring", "reduces risk")
- Interpretive connectives ("suggests", "indicates", "consistent with",
"likely", "concerning for")
- Weighing/aggregating/conclusion statements that assess overall risk or
probability
- Predictive statements or probability assessments

KEEP UNCHANGED as much as possible:

- The overall structure, section ordering, and formatting (including think
tags and ### headers)
- All factual clinical content: demographics, diagnoses, lab values with
numbers, vitals, medications, procedures, dates, and other concrete data
points
- The exact wording of factual statements - do NOT rephrase facts that contain
no interpretation
- Section headers - keep them but strip interpretive framing from bullet
points beneath them
- Bullet point structure and ordering

IMPORTANT:

- Make the FEWEST changes necessary. If a bullet point is purely factual,
leave it verbatim.
- If a bullet mixes fact + interpretation, keep the fact and delete only
the interpretive clause. Example: "Hyperglycemia (glucose 202 mg/dL)
indicating diabetes or stress hyperglycemia (associated with worse outcomes)"
-> "Hyperglycemia (glucose 202 mg/dL)."
- If a bullet is purely interpretive with no factual content, delete it
entirely.
- Do NOT add any new information, reorganize sections, or change the summary's
structure.
- The section "WEIGHING AND AGGREGATING THE EVIDENCE" (section 5) should
be ENTIRELY removed - delete the header and all its content. It is purely
interpretive.
```

Figure 20: Prompt used with GPT-5-mini for stripping away interpretive language from Local-Rubric representation (*i.e.*, modifying x^{rubric}).

G Full global rubric examples

G.1 Full global rubric for the hypertension onset prediction task

RUBRIC INSTRUCTIONS FOR TASK: HYPERTENSION

Rubric purpose

- Provide a reproducible, stepwise process to transform any EHR into a structured, clinical-evidence summary useful for assessing the likelihood that a patient will develop hypertension in the next year.
- The rubric standardizes what to extract, how to summarize trends and risk factors, and how to record uncertainty and provenance so downstream models or clinicians can apply consistent reasoning.

How to use this rubric

- Follow the numbered extraction and analysis steps for each new patient.
- Populate the structured template fields exactly (use units shown). If data are missing, enter “missing” and note time windows attempted.
- Do NOT make a final yes/no prediction inside the form. Instead, produce the structured summary and quantitative or qualitative risk-domain scores for downstream modeling.

A. Preparation (before extracting)

1. Define the prediction window: “next year” relative to the EHR reference date and time.
2. Define time windows to extract:
 - Very recent: last 30 days
 - Recent: 31-180 days
 - Baseline/remote: >180 days up to available history
3. Standardize units and formats:
 - Blood pressure: mmHg (systolic/diastolic)
 - Weight: kg or oz → convert to kg if numeric calculations needed
 - Height: cm or in → convert to meters
 - Labs: use usual clinical units (creatinine mg/dL, A1c %, etc.)
4. Log data sources (vitals, problem list, medications, laboratory, procedures, notes) and timestamp of extraction.

B. Step-by-step extraction & transformation procedure

Step 1 -- Demographics and baseline context

- Extract:
 - Age (years)
 - Sex / gender
 - Race / ethnicity (if available)
 - Relevant social history: tobacco (current/former/never), alcohol (heavy/regular/rare/none), illicit drug use, tobacco product types
 - Pregnancy status (current or past complications such as pre-eclampsia)
 - Baseline height and weight; calculate BMI (kg/m^2) and BMI category
- Record date of last update for each demographic item.

Step 2 -- Blood pressure (BP) data extraction and normalization

- Extract all systolic and diastolic BP values with timestamps and context (office, inpatient, ED, home, ambulatory, perioperative).
- Normalize values: remove implausible readings (document them) and ensure mmHg units.
- For each time window:
 - Compute count, mean, median, standard deviation, minimum, and maximum.
 - Identify last available BP and date.
 - Flag highest recent systolic and diastolic values with dates.
- Compute trend metrics:
 - Recent slope = $(\text{mean_recent} - \text{mean_baseline}) / \text{time}$ (mmHg per month); indicate direction only if clinically meaningful (e.g., ≥ 3 mmHg/year).
 - BP variability indicator: SD of systolic BP in recent window; flag high variability if SD > 10 mmHg.
- Categorize BP using ACC/AHA thresholds:
 - Normal ($< 120 / < 80$)
 - Elevated ($120 - 129 / < 80$)
 - Stage 1 Hypertension ($130 - 139$ or $80 - 89$)
 - Stage 2 Hypertension (≥ 140 or ≥ 90)
- If mixed, note “discordant” and list counts per category.

Step 3 -- Antihypertensive and BP-impacting medications

- Extract current and recent medications with start and stop dates if available.
- Flag antihypertensives (ACEi, ARBs, beta-blockers, diuretics, CCBs, vasodilators).
- Flag BP-raising agents (systemic corticosteroids, NSAIDs, decongestants, stimulants, calcineurin inhibitors, SNRIs, MAOIs, some oral contraceptives).
- For each flagged medication record name, dose, dates, indication, and temporal relation to BP

changes.

Step 4 -- Comorbidities associated with increased HTN risk

- Extract diagnoses and ICD codes with dates:
- Major risk: CKD, diabetes, CVD, PAD, OSA, endocrine causes, pregnancy or pre-eclampsia, obesity (BMI ≥ 30), heavy alcohol use.
- Moderate risk: hyperlipidemia, metabolic syndrome, thyroid disease, autoimmune disease with renal involvement.
- Secondary HTN clues: resistant BP, hypokalemia, episodic symptoms.
- Record first documentation date, last active date, and severity when available.

Step 5 -- Relevant laboratory data

- Extract labs with dates grouped by time window:
- Creatinine / eGFR
- Electrolytes (Na, K, HCO_3)
- Glucose, HbA1c
- Lipids
- Urine albumin or protein
- Thyroid tests
- Aldosterone/renin, cortisol, catecholamines if available
- Flag abnormal values with interpretation (e.g., eGFR < 60 ml/min).

Step 6 -- Procedures and objective testing

- Extract echocardiography, renal imaging, sleep studies, ABPM.
- Note evidence of end-organ effects (LVH, albuminuria, renal disease).

Step 7 -- Social, behavioral, and family data

- Smoking status and intensity.
- Alcohol use severity.
- Family history of HTN or early CVD.
- Adherence or socioeconomic barriers if documented.

Step 8 -- Acute confounders

- Identify acute illness, pain, surgery, sepsis, AKI, or inpatient context affecting BP interpretation.
- Avoid using isolated inpatient readings without outpatient corroboration.

Step 9 -- Domain synthesis and scoring

- For each domain, record evidence, recency, and confidence (High/Moderate/Low).
- Domains include BP phenotype, medications, metabolic risk, kidney function, secondary HTN, end-organ disease, behavior, and acute confounders.
- Assign severity (Major/Moderate/Minor) and create a domain scorecard; do NOT generate a final binary label.

Step 10 -- Evidence provenance and missing data

- Record source, timestamp, and confidence for each major item.
- Explicitly flag critical missing data (e.g., no outpatient BP in 12 months).

Step 11 -- Structured output

- Produce a standardized summary with demographics, BP summary, medications, comorbidities, labs, procedures, lifestyle, acute confounders, domain scorecard, missing data, and a 2-4 sentence neutral text summary.

Step 12 -- Guidance notes

- Recommend confirmatory testing or review where appropriate (e.g., home BP, med review, nephrology referral).
- Do not conclude final risk.

Final note to the user

- Use this rubric to populate the structured template for every patient. Do not record a final hypertension risk classification here; the output is intended for downstream models or clinician judgment.

Figure 21: Global rubric instructions for extracting structured patient profiles for 1-year hypertension diagnosis prediction.

G.2 Full global rubric for the hyponatremia lab result prediction task

RUBRIC INSTRUCTIONS FOR TASK: HYPONATREMIA LAB RESULT

PredictionDate: [Extract the 'current time' / prediction timestamp from the EHR header].

Format: YYYY-MM-DD. If not present write NA.

Patient:

- Age: [years as integer from EHR]. If not present write NA.
- Sex: [as documented: MALE / FEMALE / Other / Unknown]. If not present write NA.
- Race/Ethnicity: [as documented]. If not present write NA.

ProblemListFlags (presence and dates):

- Chronic kidney disease / End-stage renal disease (CKD/ESRD): [Yes / No]. If Yes, list documented term(s) and most recent date(s) (YYYY-MM-DD). If none write No.
- Dialysis history/procedure in record: [Yes / No]. If Yes, list procedure name(s) and most recent date(s). If none write No.
- Prior documented hyponatremia / 'hypo-osmolality and or hyponatremia': [Yes / No]. If Yes, give the documentation text and date(s). If none write No.
- Active malignancy listed in Problem List or current visits: [Yes / No]. If Yes, list malignancy type(s) and most recent date(s). If none write No.

SerumSodium_Last3 (most recent first): For up to 3 most recent serum/plasma/blood sodium measurements, extract a line per measurement in this exact format:

- YYYY-MM-DD (days_before_prediction): [value] mmol/L ; Specimen=[serum/plasma/blood] ; Setting=[ED/Inpatient/Outpatient/Lab] ; Note=[any explicit result comment if present]
If fewer than 3 measurements exist, include those available; if none write NA.
- SerumSodium_Min90:
 - Lowest documented serum/plasma/blood sodium value in the prior 90 days (value mmol/L) and date (YYYY-MM-DD). If none write NA.
- SerumOsmolality_Last3:
 - For up to 3 most recent serum osmolality measurements: YYYY-MM-DD (days_before_prediction): [value] mOsm/kg ; Setting=[as above]
If none write NA.

UrineStudies_Last3:

- For up to 3 most recent urine study sets, extract for each available element on one line:
 - YYYY-MM-DD (days_before_prediction): UrineNa=[value] mmol/L ; UrineOsm=[value] mOsm/kg ; SpecificGravity=[value] ; Setting=[ED/Inpatient/Outpatient/Lab]
- Only include elements that are present for that date. If no urine studies documented write NA.

RenalFunction:

- Most recent serum creatinine (mg/dL) and date: YYYY-MM-DD: [value] mg/dL. If none write NA.
- Most recent BUN (mg/dL) and date: YYYY-MM-DD: [value] mg/dL. If none write NA.
- Recent acute renal failure / acute kidney injury entries within 30 days: [Yes / No]. If Yes include diagnosis text and date(s). If none write No.

VolumeRelatedFindings (documented in problem lists or visit notes within past 30 days):

- Extract presence with dates for these items (list each if present as "Item: YYYY-MM-DD;"): Edema, Ascites, Hypotension (documented low BP or explicit "hypotension"), Dehydration, Vomiting, Diarrhea, Nasogastric/feeding tube, Ileostomy/colostomy, Recent large-volume paracentesis. If none of these documented in past 30 days write NA.

Medications_PotentiallyAffectingSodium (recent administrations - extract from medication list / inpatient meds / discharge meds):

- Time window: last 14 days before PredictionDate (if EHR supports more granular times use those). For each relevant med/class present include one line:
 - [YYYY-MM-DD last administration if available] : [Medication name] ; Class=[thiazide/loop diuretic / SSRI / SNRI / TCA / anticonvulsant (carbamazepine/oxcarbazepine) / NSAID / SSRI, etc.] ; Route=[oral/IV] ; Dose if documented=[text]
- If none of these medication classes documented in last 14 days write NA.
- Also include "Chronic diuretic use noted (Yes/No) and last documentation date" (e.g., long-term thiazide).

IVFluids_Last72h:

- List IV fluid administrations in last 72 hours (date/time if available) in the format:
 - YYYY-MM-DD: [fluid type as documented, e.g., D5W / D5NS / 0.9% NaCl / hypotonic saline / LR / "glucose 50 mg/mL prefills"] ; Volume if documented.
- If none documented write NA.

AcuteConditions_AssociatedWithSIADHorHyponatremia (documented within 30 days):

- For each present within 30 days, list as "Condition: YYYY-MM-DD" from problem/visit notes:
 - Pulmonary infection / pneumonia / pulmonary disease
 - CNS disorder (stroke, hemorrhage, encephalopathy)
 - Sepsis / severe infection

- Recent major surgery / Postoperative state
- Pain / Severe nausea (if explicitly documented)
- Malignancy active (if not already in ProblemListFlags)

If none documented write NA.

RecentProcedures_Chemotherapy_Transfusion (last 30 days):

- List any of: major surgery, chemotherapy, recent blood transfusion, paracentesis, TPN (total parenteral nutrition), plasmapheresis, hemodialysis - format:
- YYYY-MM-DD: [procedure name / chemo agent e.g., paclitaxel] ; Notes=[if available]

If none write NA.

Glucose_Last3:

- Up to 3 most recent serum/plasma or point-of-care glucose values (most recent first):
- YYYY-MM-DD: [value] mg/dL ; Type=[serum/plasma/glucometer] ;
- Setting=[ED/Inpatient/Outpatient]

If none write NA.

SerumProteinOrLipidExtremes:

- If very high triglycerides or abnormal total protein/albumin documented close to sodium measurement, extract:
- YYYY-MM-DD: Triglycerides=[value] mg/dL ; TotalProtein=[value] g/dL ; Albumin=[value] g/dL

If none documented write NA.

PriorHyponatremiaHistory:

- Any historical low sodium episodes before 90 days (brief): list lowest prior value and date(s) or write NA.

LabQualityNotes:

- Any documented lab-quality flags on sodium measurement (e.g., hemolysis, lipemia, "evacuated blood collection tube" note, specimen issues): extract verbatim note and date(s). If none write NA.

RelevantVitalSigns_NearMostRecentSodium:

- From the same encounter as the most recent sodium (if identifiable), extract: systolic/diastolic BP (mmHg), heart rate (bpm), and whether on oxygen or dialysis that encounter. Format:
- Date: YYYY-MM-DD ; SBP=[value] ; DBP=[value] ; HR=[value] ; Oxygen=[yes/no] with O2 sat if given ; DialysisThisEncounter=[yes/no]

If not available write NA.

FreeText_Findings_Cues:

- Extract any verbatim phrases (short quotes) that explicitly mention hyponatremia-related language in notes or problem list (e.g., "hyponatremia", "hypo-osmolality", "SIADH", "hypotonic fluids", "low sodium") with the date and the note type. Format:
- YYYY-MM-DD ; Source=[ProblemList/VisitNote/LabComment] ; Text="[exact phrase]"

If none write NA.

DataCompleteness:

- For each of the following categories indicate [Present / Absent / Not documented]: Serum sodium labs, urine sodium/osmolality, serum osmolality, recent meds list, dialysis record, IV fluids record, creatinine/BUN. Example: SerumSodium: Present ; UrineSodium: Absent ; etc.

ExtractionRules / Formatting Rules (must follow exactly):

- Always extract facts only; do not add interpretation, risk assessment, or predictions.
- Dates: use YYYY-MM-DD as in EHR; if EHR provides relative days include "(N days before prediction)" after date.
- When multiple values on same date, include all values separated by ";".
- If an item not found anywhere in the EHR, write exactly "NA".
- Keep each field on a single line (except the repeated-measure lists which may have up to three lines as specified).
- Use units exactly as specified (mmol/L for Na and UrineNa; mOsm/kg for osmolality; mg/dL for glucose/BUN/creatinine; mg/dL for triglycerides).
- Do not synthesize or infer ranges; extract only documented numeric values and verbatim text.

EndOfTemplate.

Figure 22: Global rubric instructions for extracting structured patient profiles for hyponatremia lab result prediction (abnormal vs. normal).